

Neural Networks Training

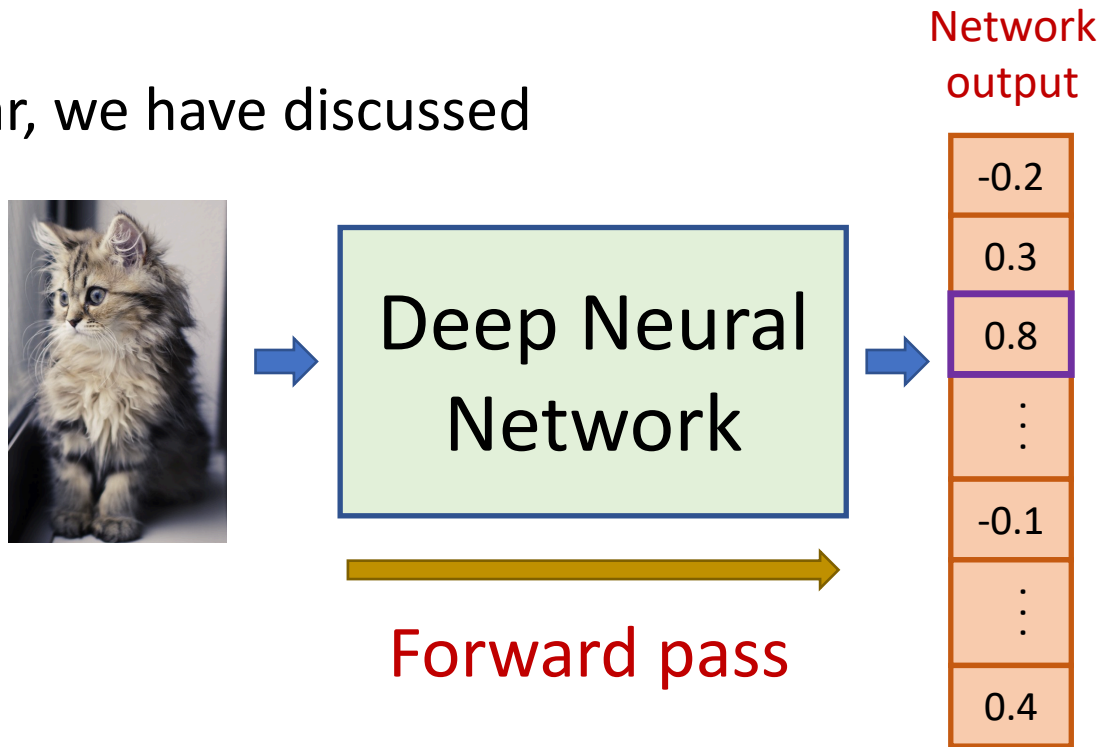


Guosheng Hu

Source from Farnoosh Heidarivincheh

Network Training

So far, we have discussed

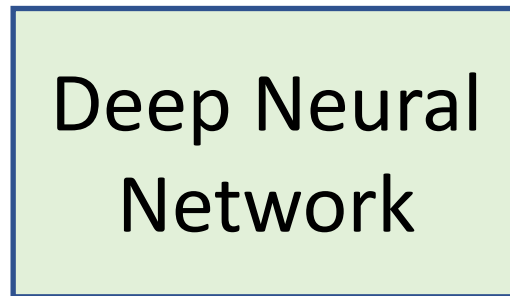


Optimisation during training

1. It makes a prediction using its current parameter set (weights).
2. We give it a feedback of how good its prediction was.
3. It updates (refines) its parameter set according to our feedback

Network Training

So far, we have discussed



Network
output

-0.2
0.3
0.8
⋮
-0.1
⋮
0.4

Cross-entropy
Loss function



Ground
truth

0	dog
0	tiger
0	ostrich
⋮	
1	Cat
⋮	
0	horse

Optimisation during training

1. It makes a prediction using its current parameter set (weights).
2. We give it a feedback of how good its prediction was.
3. It updates (refines) its parameter set according to our feedback

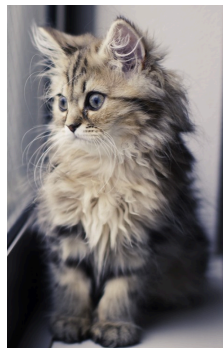
Outline

Network Training

- Gradient Descent
- Back Propagation

Network Training (Gradient Descent)

Network Training



Deep Neural
Network



Network
output

-0.2
0.3
0.8
⋮
-0.1
⋮
0.4

Ground
truth

0	dog
0	tiger
0	ostrich
⋮	
1	Cat
⋮	
0	horse

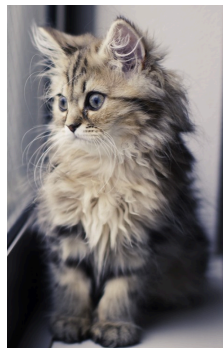
Our objective is to find an optimal parameter set ($\mathbf{W}$) which minimises our loss function

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \sum_{n=1}^N L(f(\mathbf{x}_n; \mathbf{W}), \mathbf{g})$$

Optimisation during training

1. It makes a prediction using its current parameter set (weights).
2. We give it a feedback of how good its prediction was.
3. It updates (refines) its parameter set according to our feedback

Network Training



Deep Neural
Network



Network
output

-0.2
0.3
0.8
⋮
-0.1
⋮
0.4

Ground
truth

0	dog
0	tiger
0	ostrich
⋮	
1	Cat
⋮	
0	horse

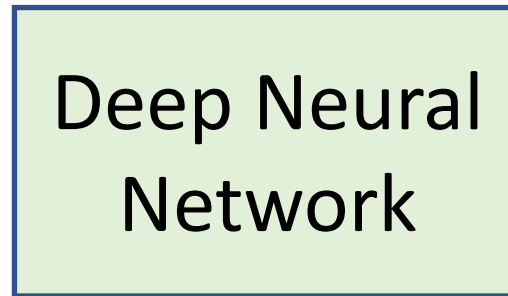
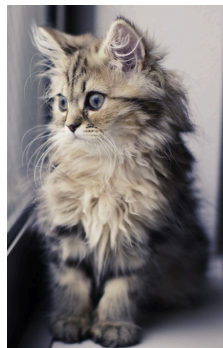
Our objective is to find an optimal parameter set ($\mathbf{W}$) which minimises our loss function

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \sum_{n=1}^N L(f(\mathbf{x}_n; \mathbf{W}), \mathbf{g})$$

Optimisation during training

1. It makes a prediction using its current parameter set (weights).
2. We give it a feedback of how good its prediction was.
3. It updates (refines) its parameter set according to our feedback

Network Training



Network
output

-0.2
0.3
0.8
⋮
-0.1
⋮
0.4

Ground
truth

0	dog
0	tiger
0	ostrich
⋮	
1	Cat
⋮	
0	horse

Our objective is to find an optimal parameter set ($\mathbf{W}$) which minimises our loss function

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \sum_{n=1}^N L(f(\mathbf{x}_n; \mathbf{W}), \mathbf{g})$$
$$= \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$

Optimisation during training

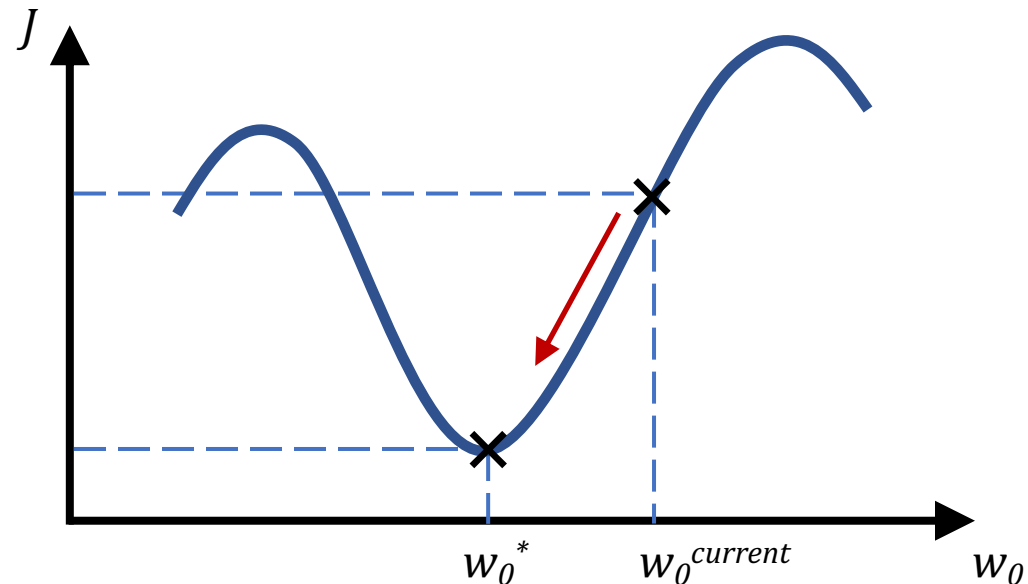
1. It makes a prediction using its current parameter set (weights).
2. We give it a feedback of how good its prediction was.
3. It updates (refines) its parameter set according to our feedback

Network Training

- We start from our current $\mathbf{W}$
- We need to move towards the direction which minimises the loss

$$\begin{aligned}\mathbf{W}^* &= \operatorname{argmin}_{\mathbf{W}} \sum_{n=1}^N L(f(\mathbf{x}_n; \mathbf{W}), \mathbf{g}) \\ &= \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})\end{aligned}$$

Note: $\mathbf{W}$ is the aggregation of several multi-dimensional matrices



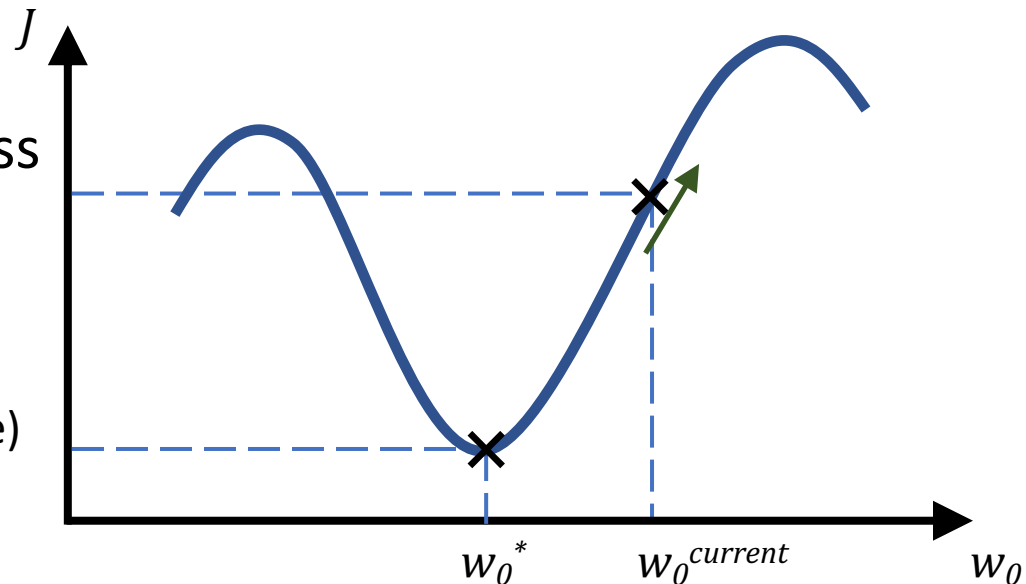
Network Training

- We start from our current $\mathbf{W}$
- We need to move towards the direction which minimises the loss

➡ **Gradient Descent (GD)**

We take the gradient of the loss
w.r.t the current $\mathbf{W}$

(The loss function must be differentiable)



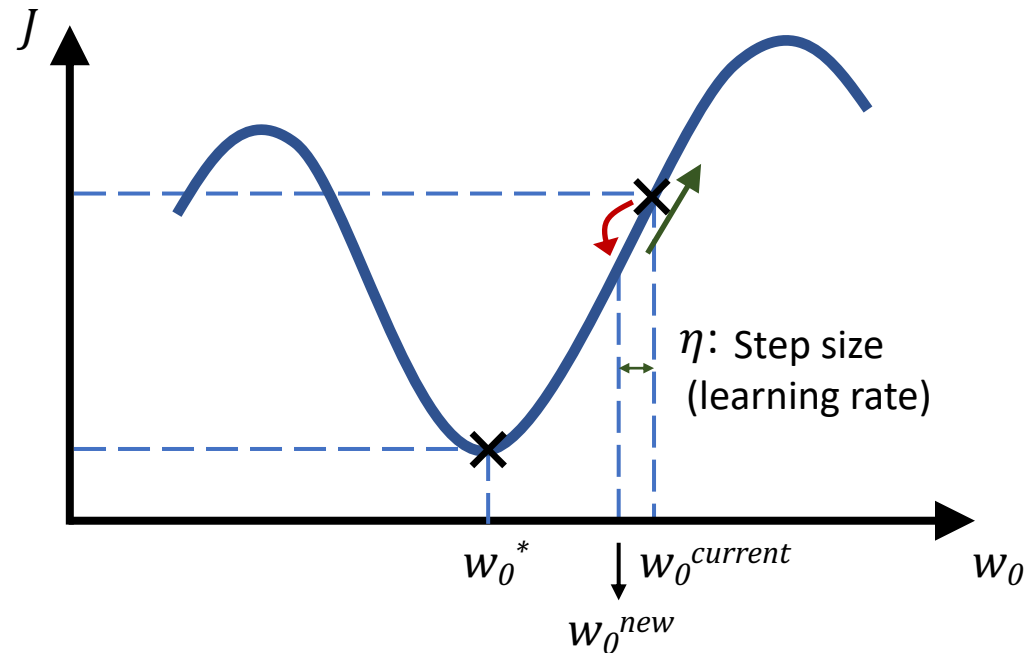
Network Training

- We start from our current $\mathbf{W}$
- We need to move towards the direction which minimises the loss

➡ Gradient Descent (GD)

We step in the opposite direction of the gradient

$$\mathbf{W}^{new} = \mathbf{W}^{current} - \eta \frac{\partial J}{\partial \mathbf{W}}$$

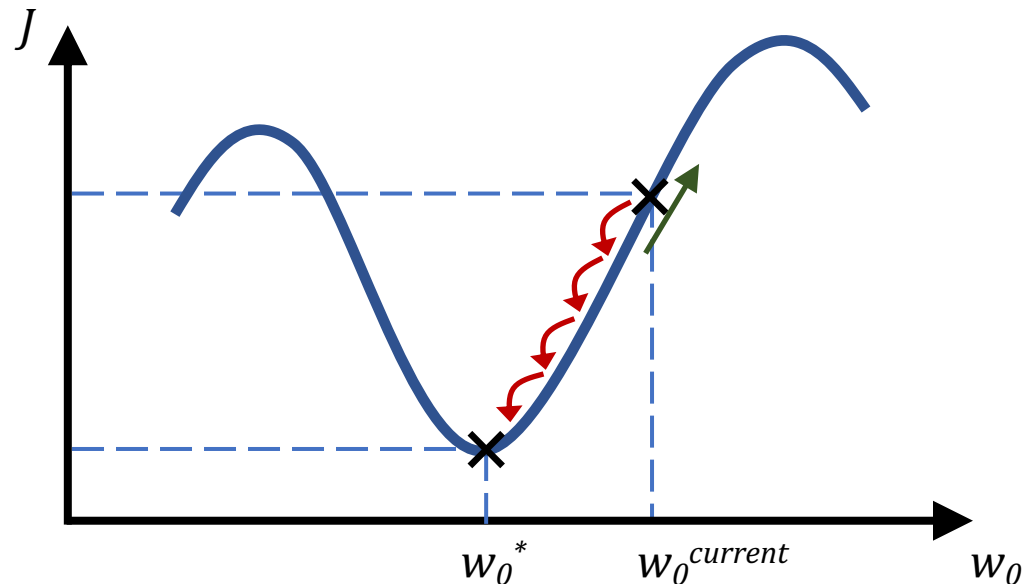


Network Training

- We start from our current $\mathbf{W}$
- We need to move towards the direction which minimises the loss

➡ **Gradient Descent (GD)**

We repeat until convergence



Network Training

Stochastic Gradient Descent (SGD)

- GD is expensive to be computed on the whole training data (X)
- We can use a single data point, e.g. one image
- This data point is selected randomly

$$\mathbf{W}^{new} = \mathbf{W}^{current} - \eta \frac{\partial J(\mathbf{W}; \mathbf{x}_i)}{\partial \mathbf{W}}$$

where, $\mathbf{x}_i \in X$

Network Training

Mini-Batch Gradient Descent

- We divide our training set into mini-batches
- Each mini-batch contains B data points

$$\mathbf{W}^{new} = \mathbf{W}^{current} - \eta \frac{\partial J(\mathbf{W}; \mathbf{x}_{Batch})}{\partial \mathbf{W}}$$

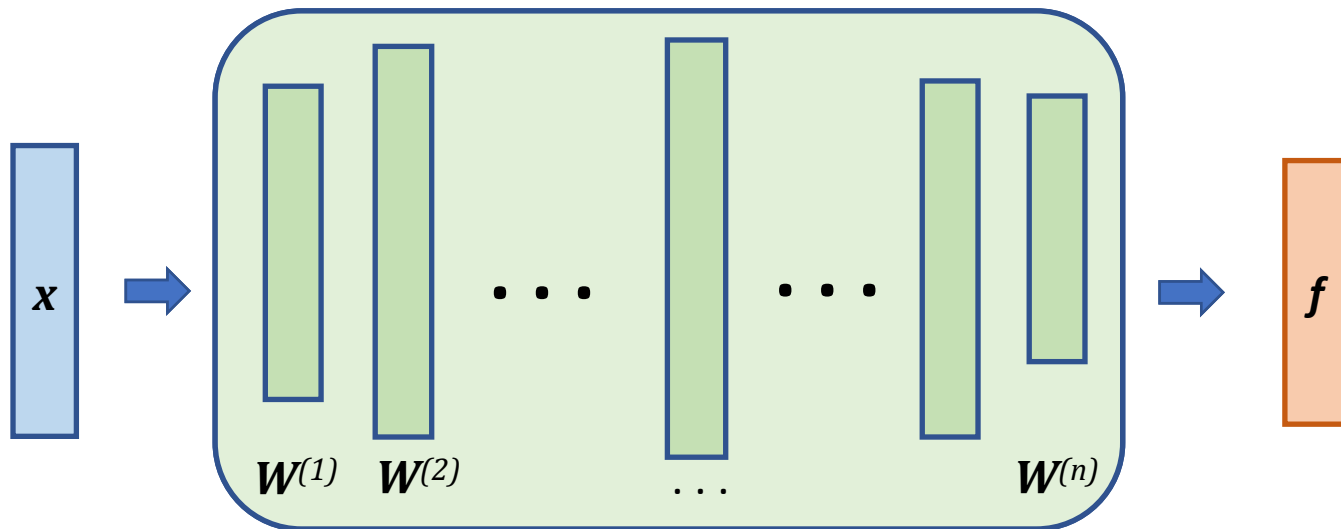
where, $\mathbf{x}_{Batch} \subset \mathbf{X}$ and $|\mathbf{x}_{Batch}| = B$
↓
(Batch size)

Network Training (Back Propagation)

Network Training

- $\mathbf{W}$ is the aggregation of several multi-dimensional matrices $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}, \dots, \mathbf{W}^{(n)}$
- The network input goes through *layers* one after another

How should we compute the loss gradient w.r.t. $\mathbf{W}$ in all those layers?

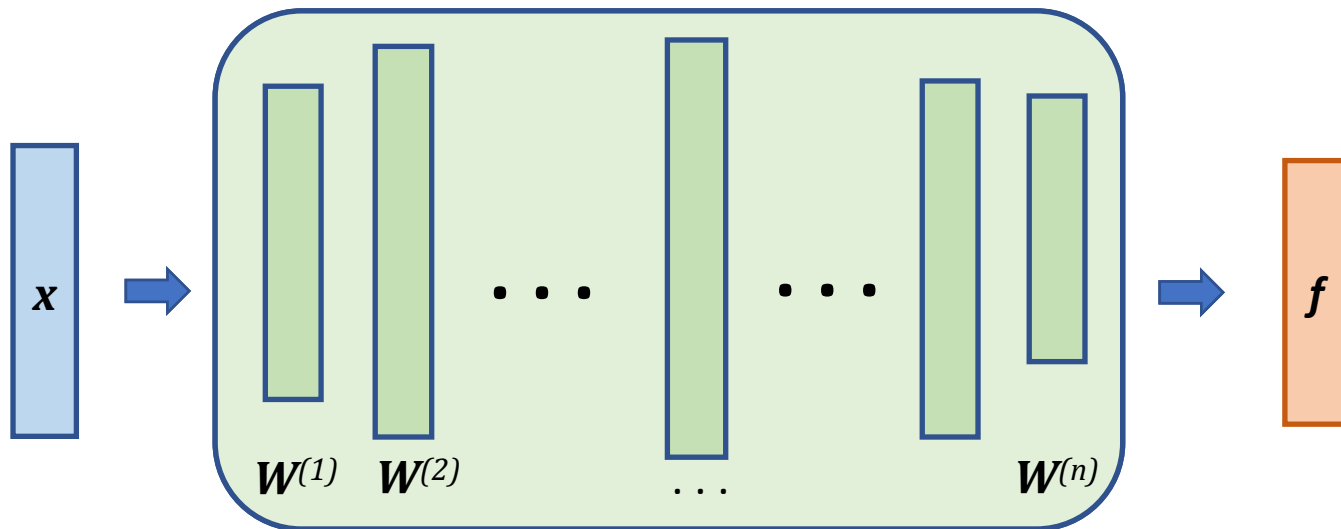


Network Training

- $\mathbf{W}$ is the aggregation of several multi-dimensional matrices $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}, \dots, \mathbf{W}^{(n)}$
- The network input goes through *layers* one after another

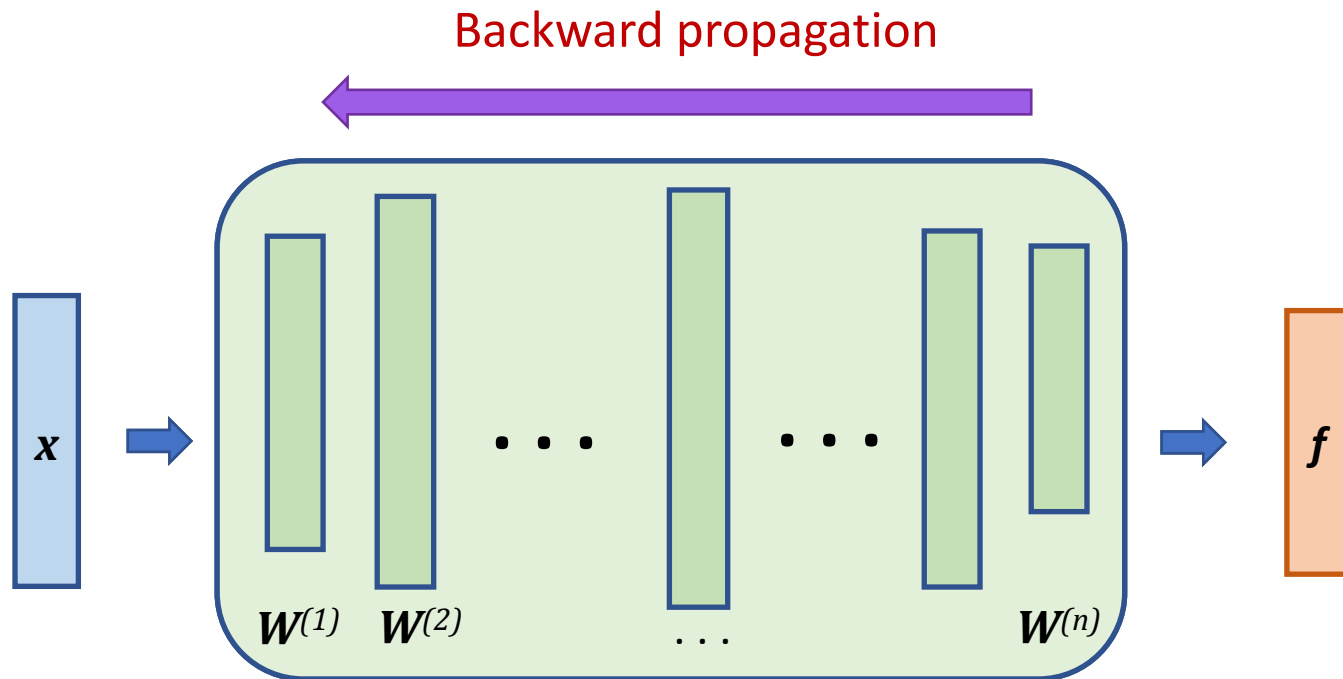
How should we compute the loss gradient w.r.t. $\mathbf{W}$ in all those layers?

Back Propagation



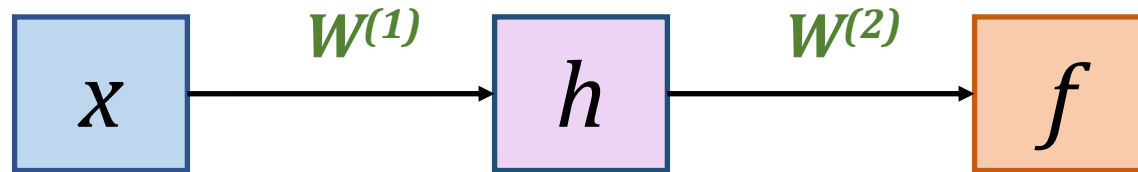
Network Training

Back Propagation (BP) uses **chain rule** recursively to back-propagate our gradient calculations **layer by layer** from **right to left**



Network Training

Example: 2 neurons with 1D input



$$W^{(1)} = [w_1, w_{b1}] = [3, -1]$$

$$W^{(2)} = [w_2, w_{b2}] = [2, 4]$$

$$x = -2$$

We want to calculate $\frac{\partial f}{\partial w_1}, \frac{\partial f}{\partial w_{b1}}, \frac{\partial f}{\partial w_2}$ and $\frac{\partial f}{\partial w_{b2}}$

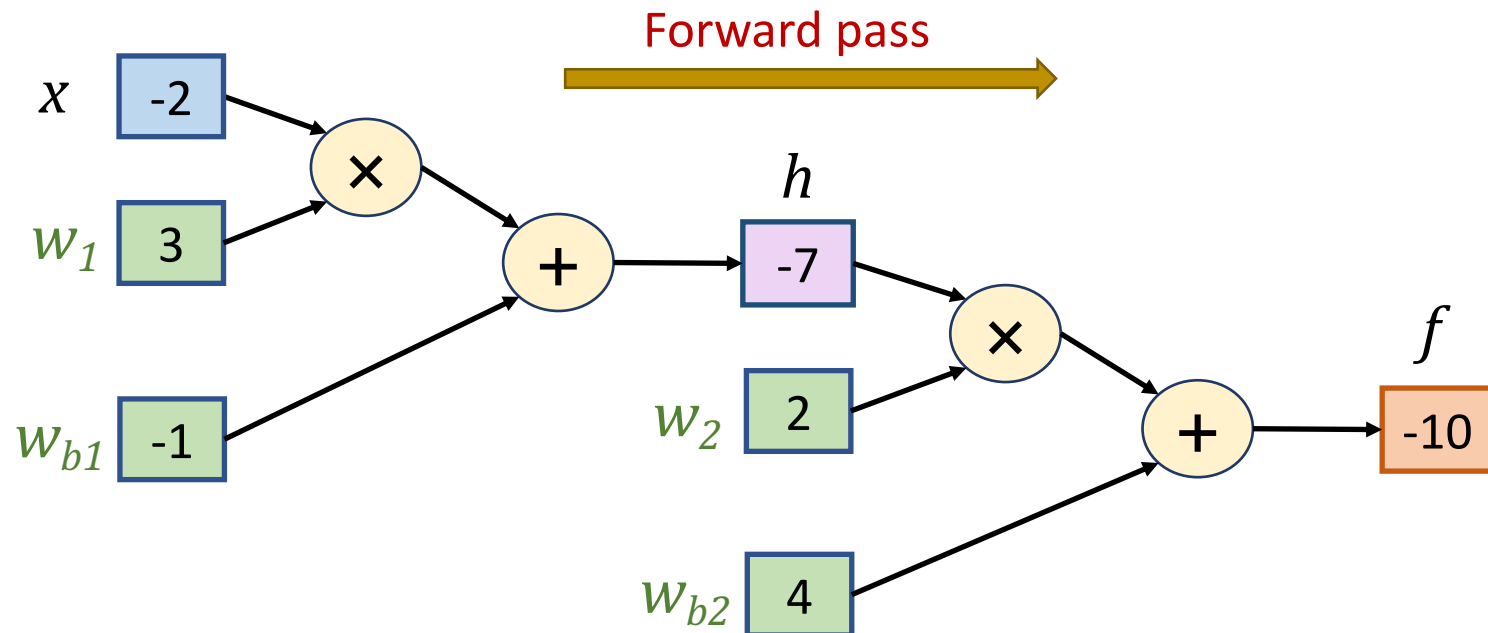
Network Training

Example: 2 neurons with 1D input

$$h = xw_1 + w_{b1}$$

(For simplicity, we remove the activation and loss functions)

$$f = hw_2 + w_{b2}$$

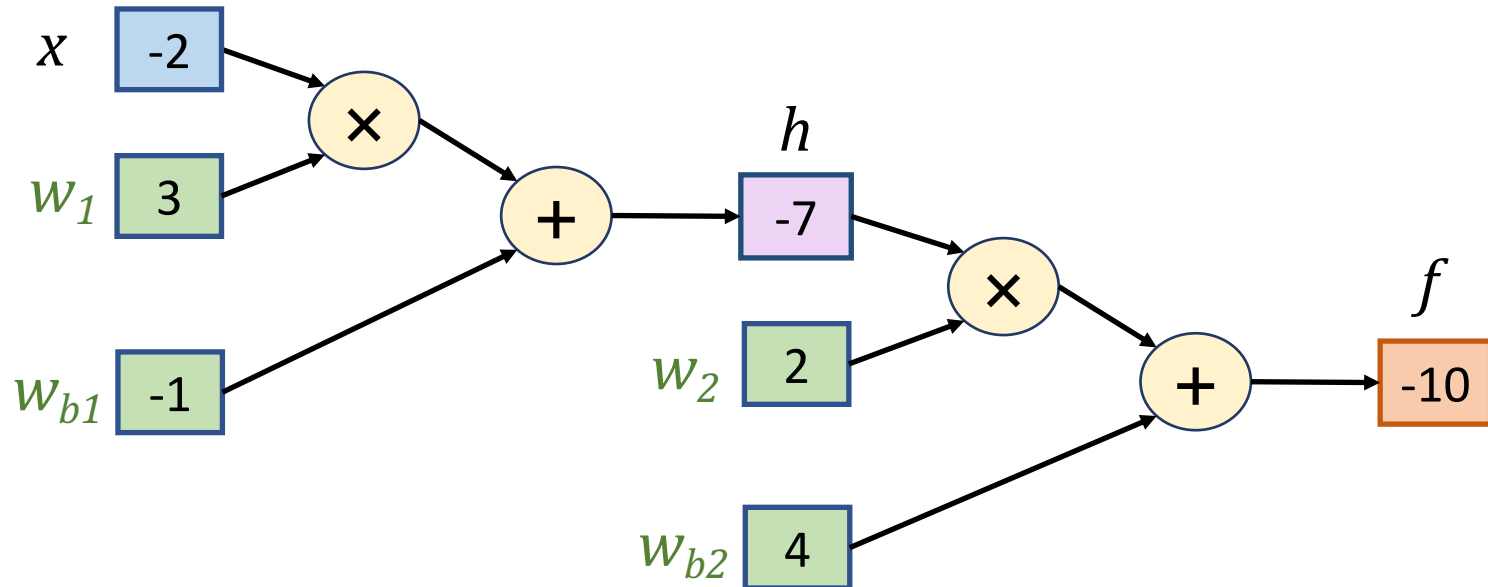


Network Training

Example: 2 neurons with 1D input

$$h = xw_1 + w_{b1} = -2 \times 3 - 1 = -7$$

$$f = hw_2 + w_{b2} = -7 \times 2 + 4 = -10$$

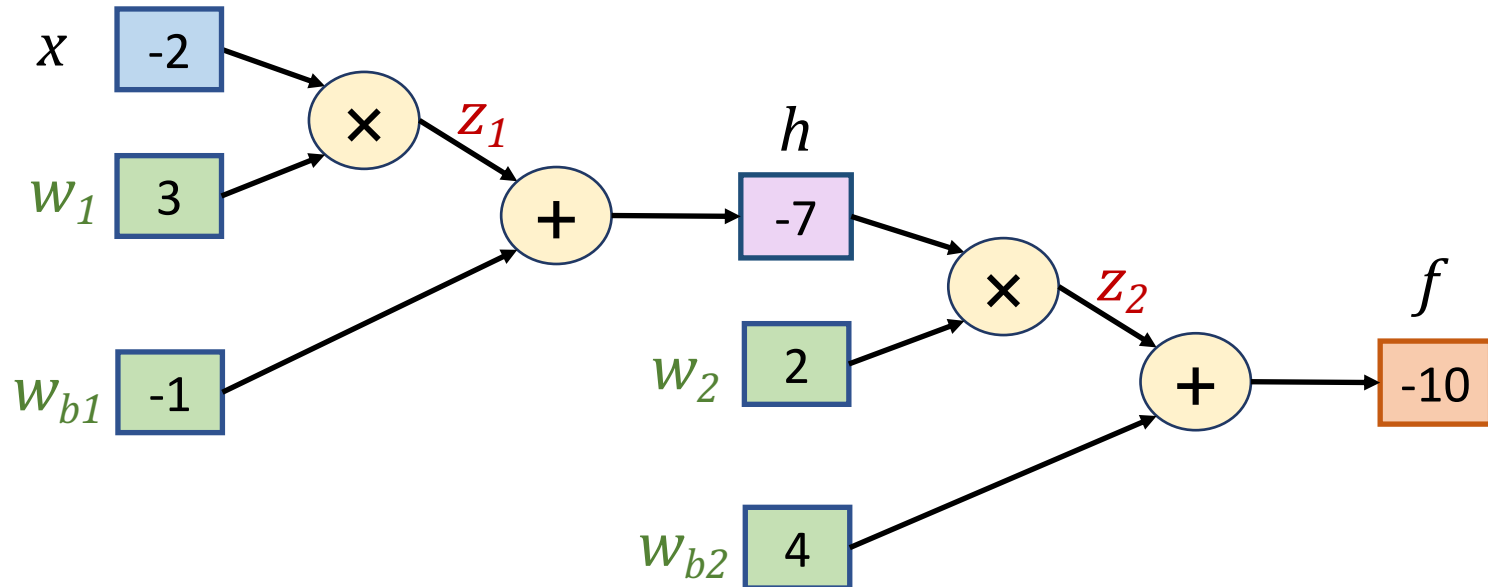


Network Training

Example: 2 neurons with 1D input

$$h = \overset{z_1}{xw_1} + w_{b1} = -2 \times 3 - 1 = -7$$

$$f = \overset{z_2}{hw_2} + w_{b2} = -7 \times 2 + 4 = -10$$



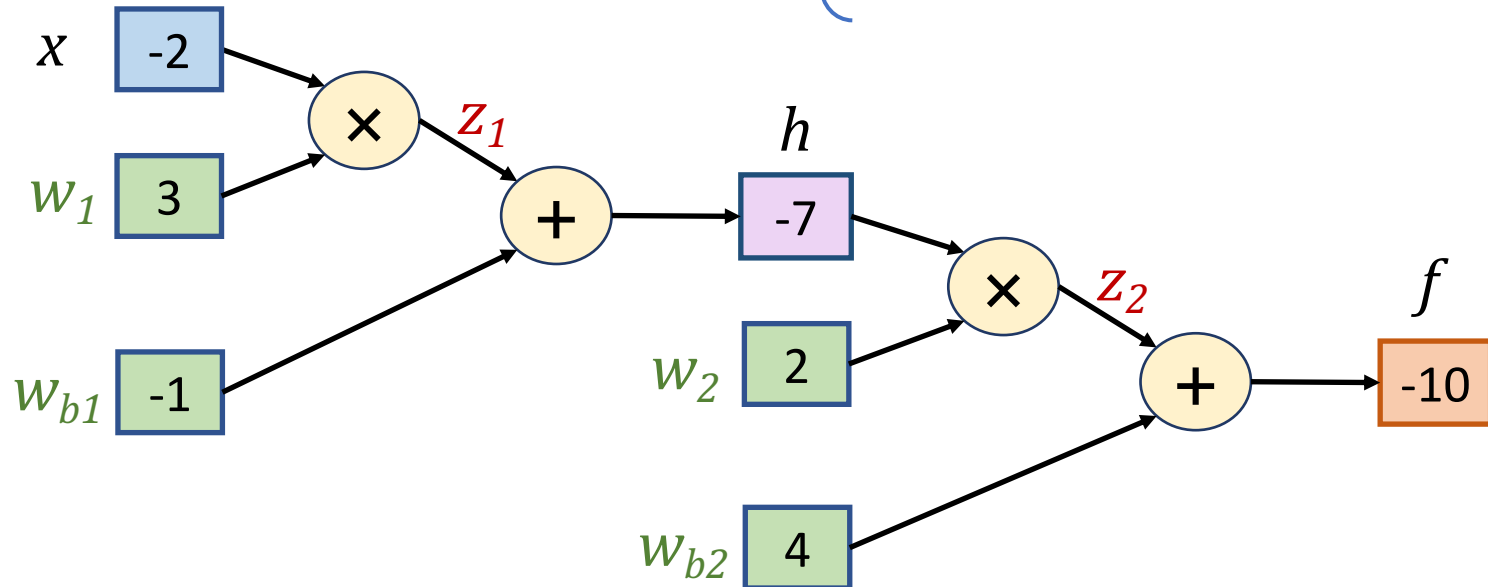
Network Training

Example: 2 neurons with 1D input

$$h = xw_1 + w_{b1} = -2 \times 3 - 1 = -7$$

$$f = hw_2 + w_{b2} = -7 \times 2 + 4 = -10$$

$$\left\{ \begin{array}{l} \frac{\partial f}{\partial w_{b2}} = 1 \\ \frac{\partial f}{\partial w_2} = \frac{\partial f}{\partial z_2} \times \frac{\partial z_2}{\partial w_2} = 1 \times h = -7 \end{array} \right.$$



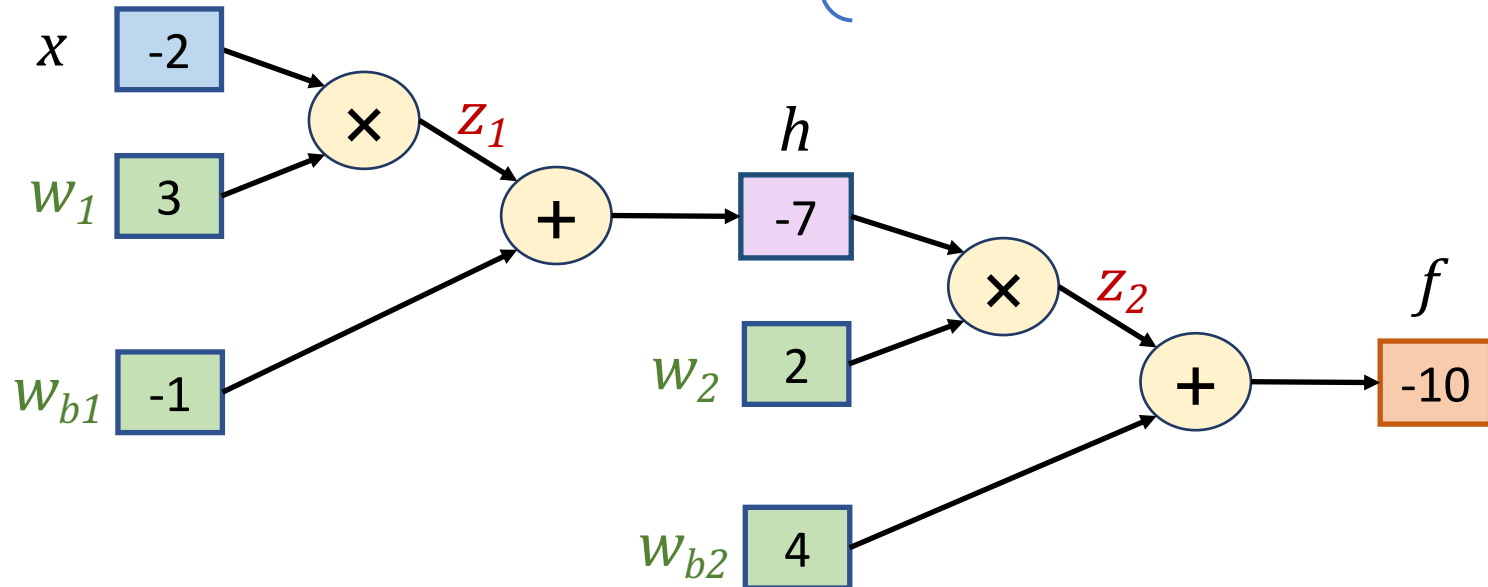
Network Training

Example: 2 neurons with 1D input

$$h = xw_1 + w_{b1} = -2 \times 3 - 1 = -7$$

$$f = hw_2 + \underline{w_{b2}} = -7 \times 2 + 4 = -10$$

$$\left\{ \begin{array}{l} \frac{\partial f}{\partial w_{b2}} = 1 \\ \frac{\partial f}{\partial w_2} = \frac{\partial f}{\partial z_2} \times \frac{\partial z_2}{\partial w_2} = 1 \times h = -7 \end{array} \right.$$



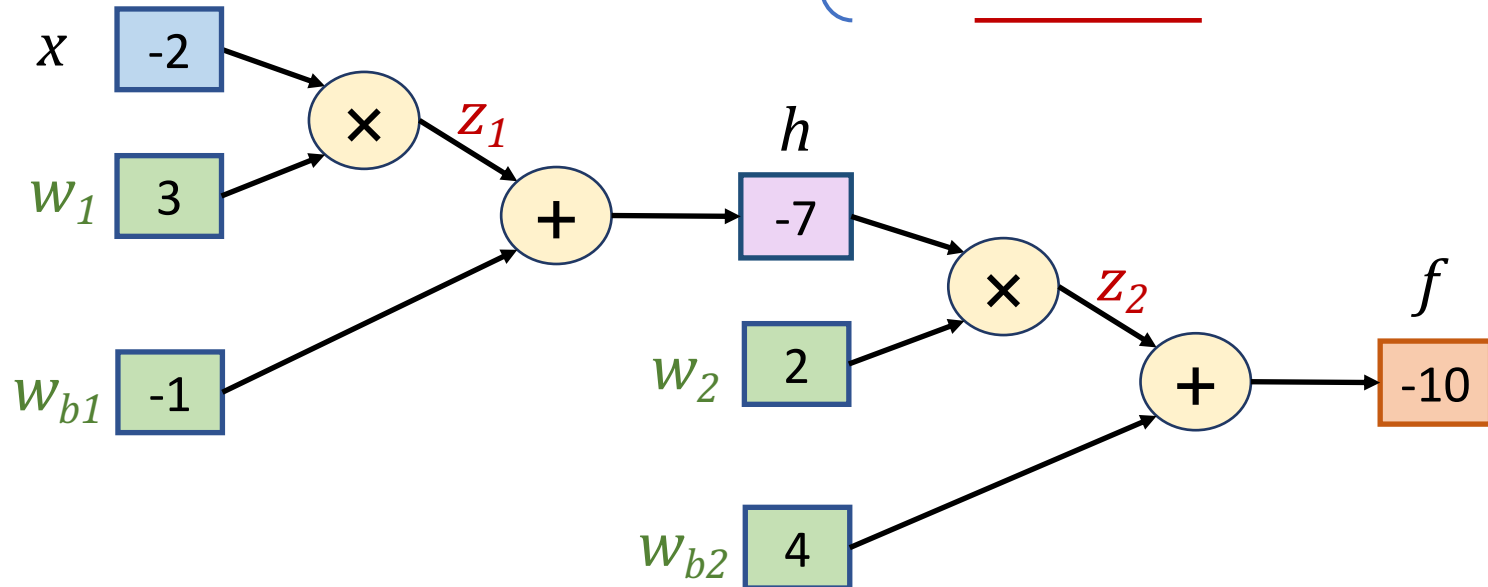
Network Training

Example: 2 neurons with 1D input

$$h = xw_1 + w_{b1} = -2 \times 3 - 1 = -7$$

$$f = \underbrace{hw_2}_{z_2} + w_{b2} = -7 \times 2 + 4 = -10$$

$$\left\{ \begin{array}{l} \frac{\partial f}{\partial w_{b2}} = 1 \\ \frac{\partial f}{\partial w_2} = \frac{\partial f}{\partial z_2} \times \frac{\partial z_2}{\partial w_2} = 1 \times h = -7 \end{array} \right.$$



Network Training

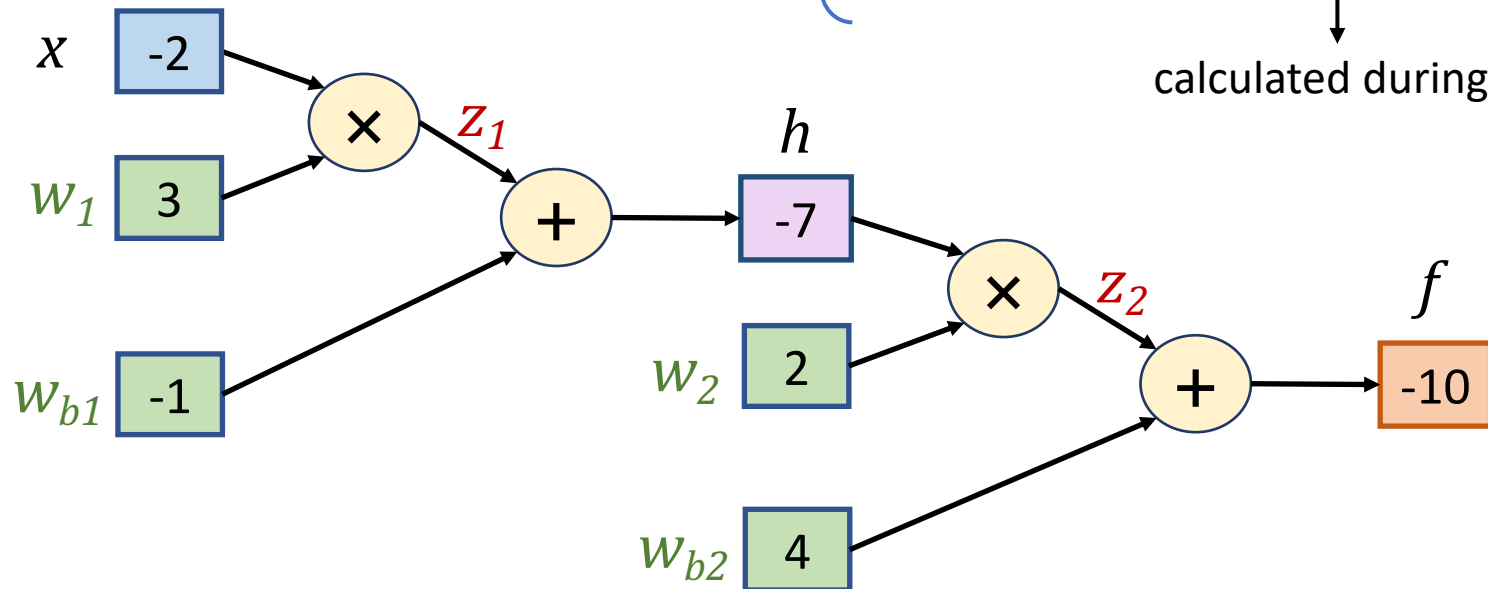
Example: 2 neurons with 1D input

$$h = xw_1 + w_{b1} = -2 \times 3 - 1 = -7$$

$$f = hw_2 + w_{b2} = -7 \times 2 + 4 = -10$$

$$\left\{ \begin{array}{l} \frac{\partial f}{\partial w_{b2}} = 1 \\ \frac{\partial f}{\partial w_2} = \frac{\partial f}{\partial z_2} \times \frac{\partial z_2}{\partial w_2} = 1 \times h = -7 \end{array} \right.$$

calculated during FP



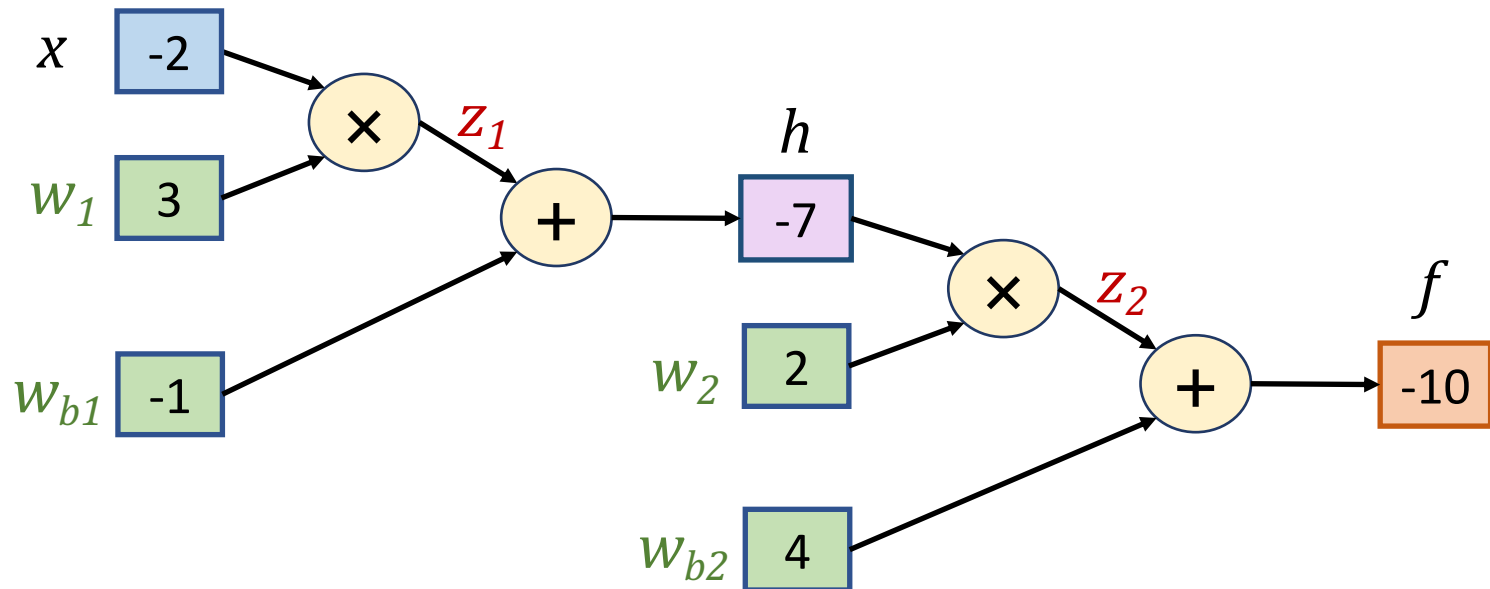
Network Training

Example: 2 neurons with 1D input

$$h = xw_1 + w_{b1} = -2 \times 3 - 1 = -7$$

$$f = hw_2 + w_{b2} = -7 \times 2 + 4 = -10$$

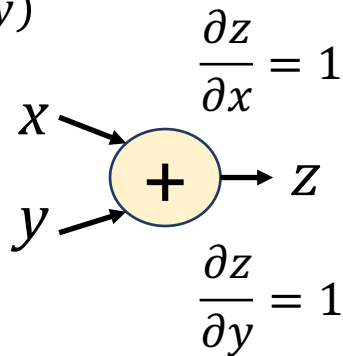
$$\left\{ \begin{aligned} \frac{\partial f}{\partial w_{b1}} &= \frac{\partial f}{\partial z_2} \times \frac{\partial z_2}{\partial h} \times \frac{\partial h}{\partial w_{b1}} \\ &= 1 \times w_2 \times 1 = 2 \\ \frac{\partial f}{\partial w_1} &= \frac{\partial f}{\partial z_2} \times \frac{\partial z_2}{\partial h} \times \frac{\partial h}{\partial z_1} \times \frac{\partial z_1}{\partial w_1} \\ &= 1 \times w_2 \times 1 \times x = -4 \end{aligned} \right.$$



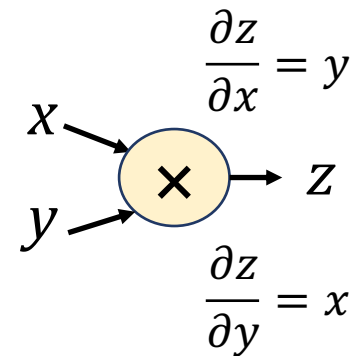
Network Training

- BP computes the partial derivatives of each function **locally** w.r.t. the inputs
- Partial derivatives from the previous layers are also multiplied using chain rule

$$(z = x + y)$$

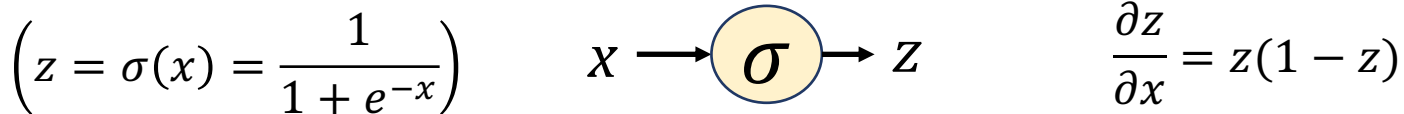
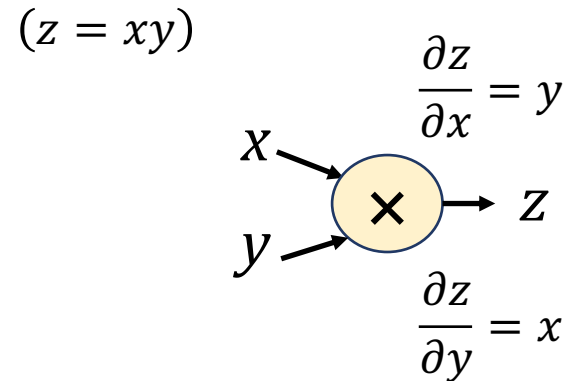
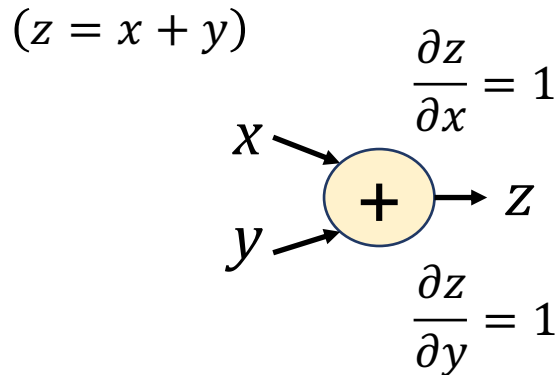


$$(z = xy)$$



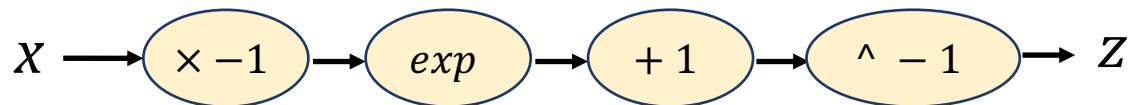
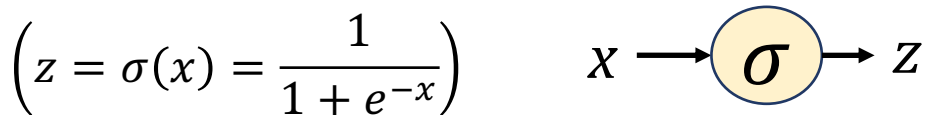
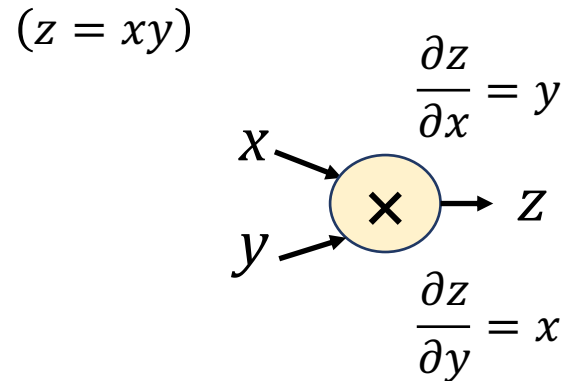
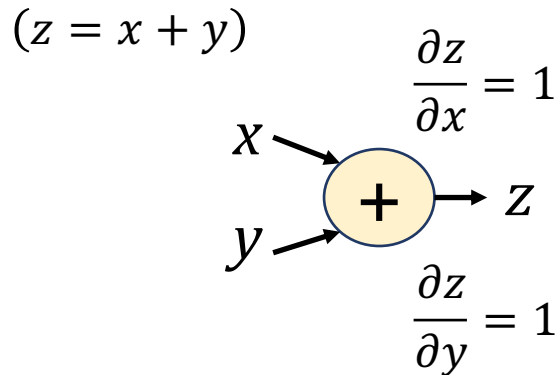
Network Training

- BP computes the partial derivatives of each function **locally** w.r.t. the inputs
- Partial derivatives from the previous layers are also multiplied using chain rule



Network Training

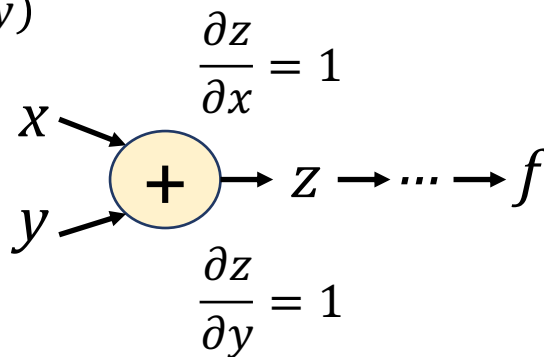
- BP computes the partial derivatives of each function **locally** w.r.t. the inputs
- Partial derivatives from the previous layers are also multiplied using chain rule



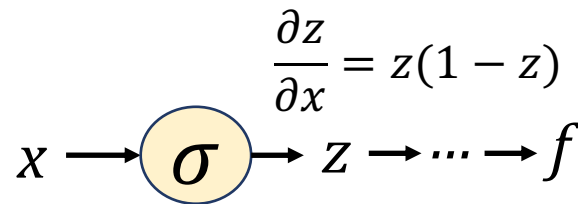
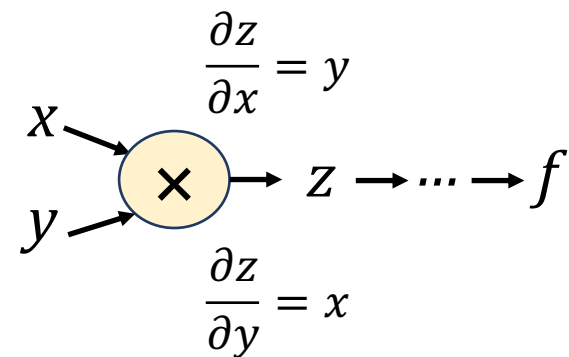
Network Training

- BP computes the partial derivatives of each function **locally** w.r.t. the inputs
- Partial derivatives from the previous layers are also multiplied using chain rule

$$(z = x + y)$$



$$(z = xy)$$

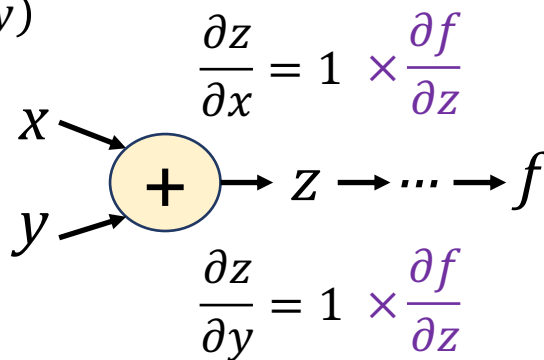


$$\left(z = \sigma(x) = \frac{1}{1 + e^{-x}} \right)$$

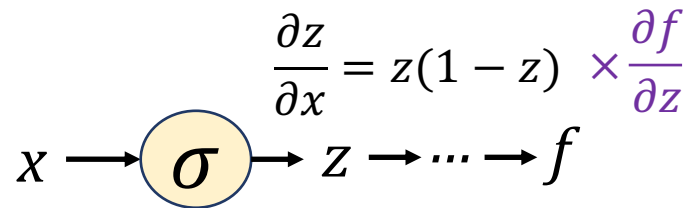
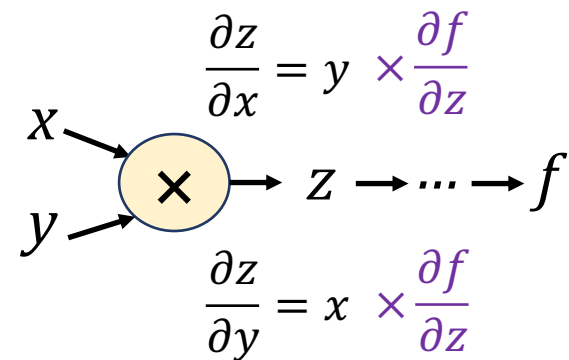
Network Training

- BP computes the partial derivatives of each function **locally** w.r.t. the inputs
- Partial derivatives from the previous layers are also multiplied using chain rule

$$(z = x + y)$$

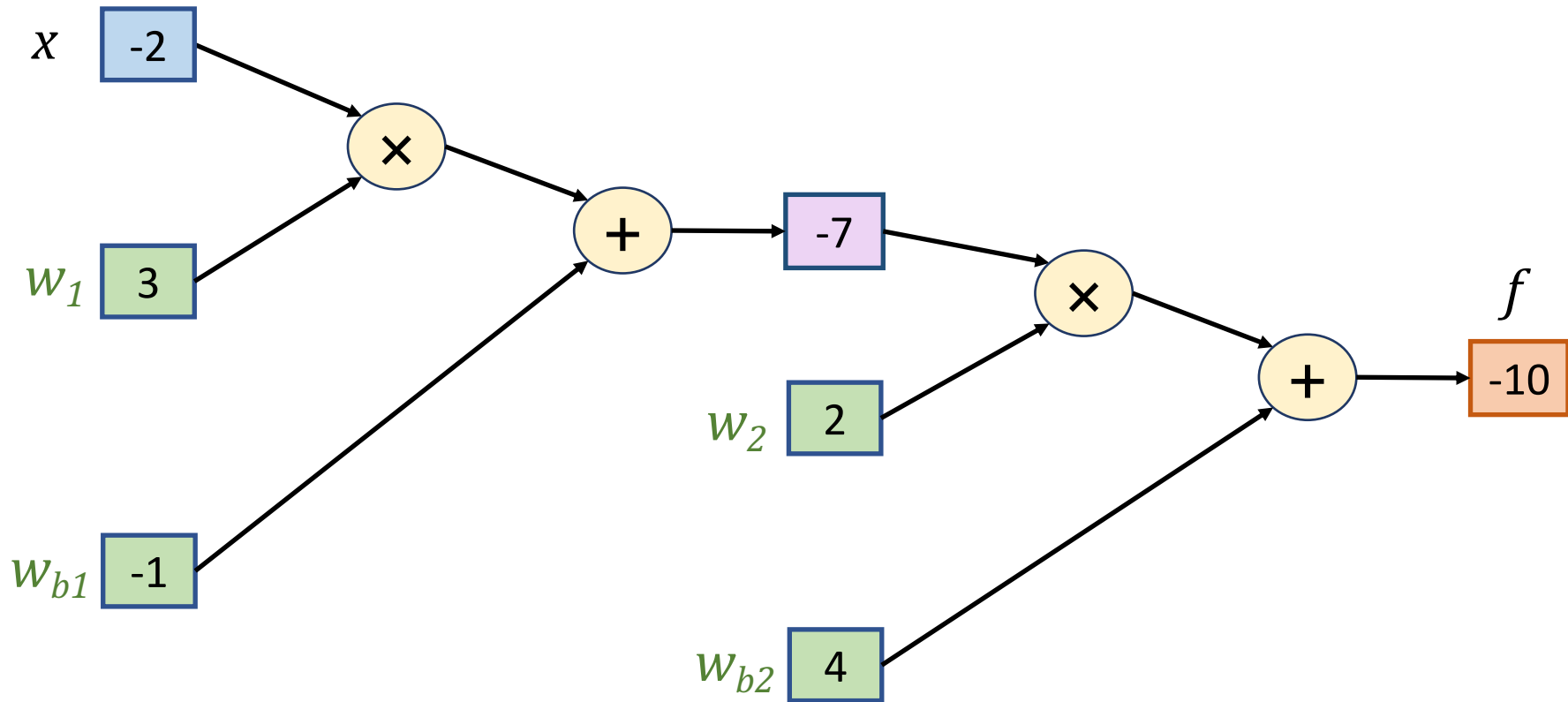


$$(z = xy)$$

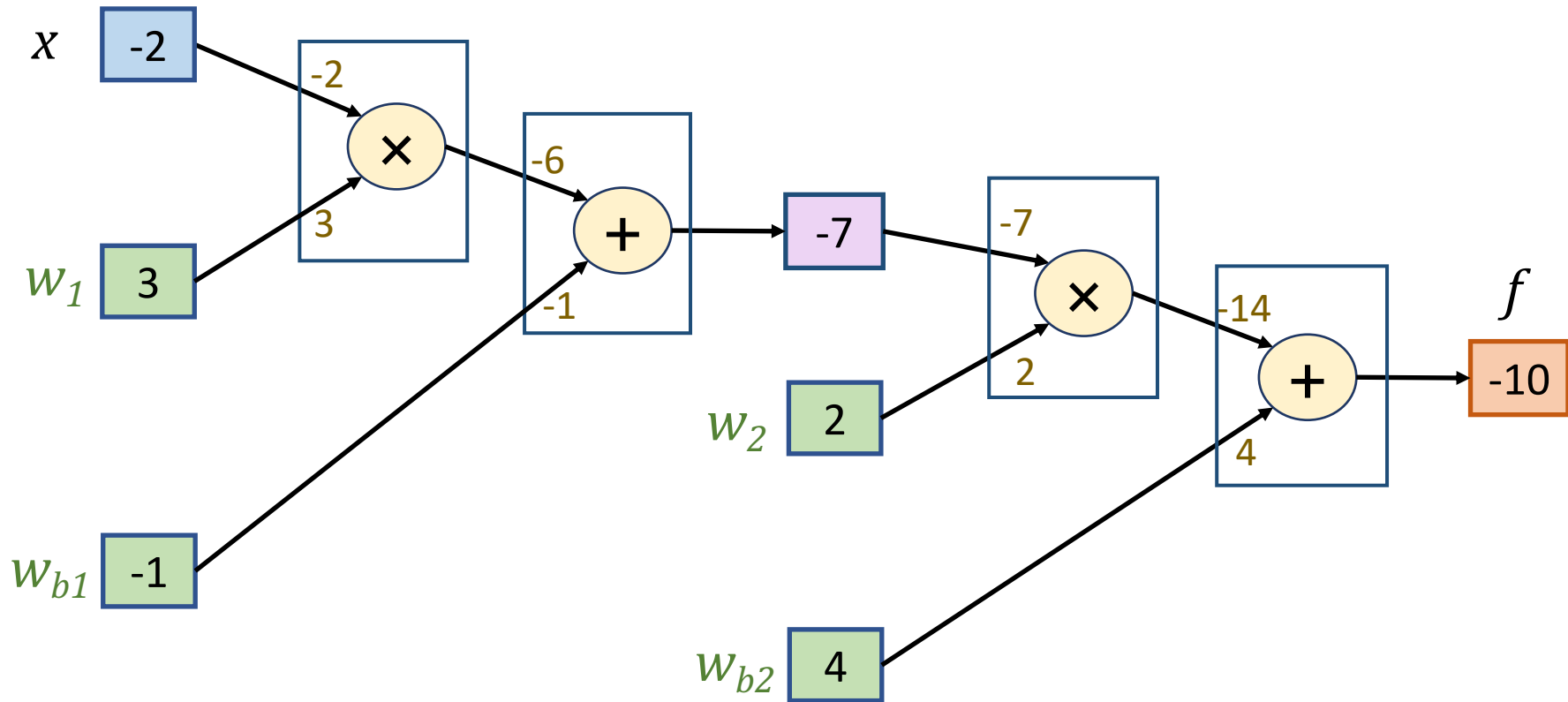


$$\left(z = \sigma(x) = \frac{1}{1 + e^{-x}} \right)$$

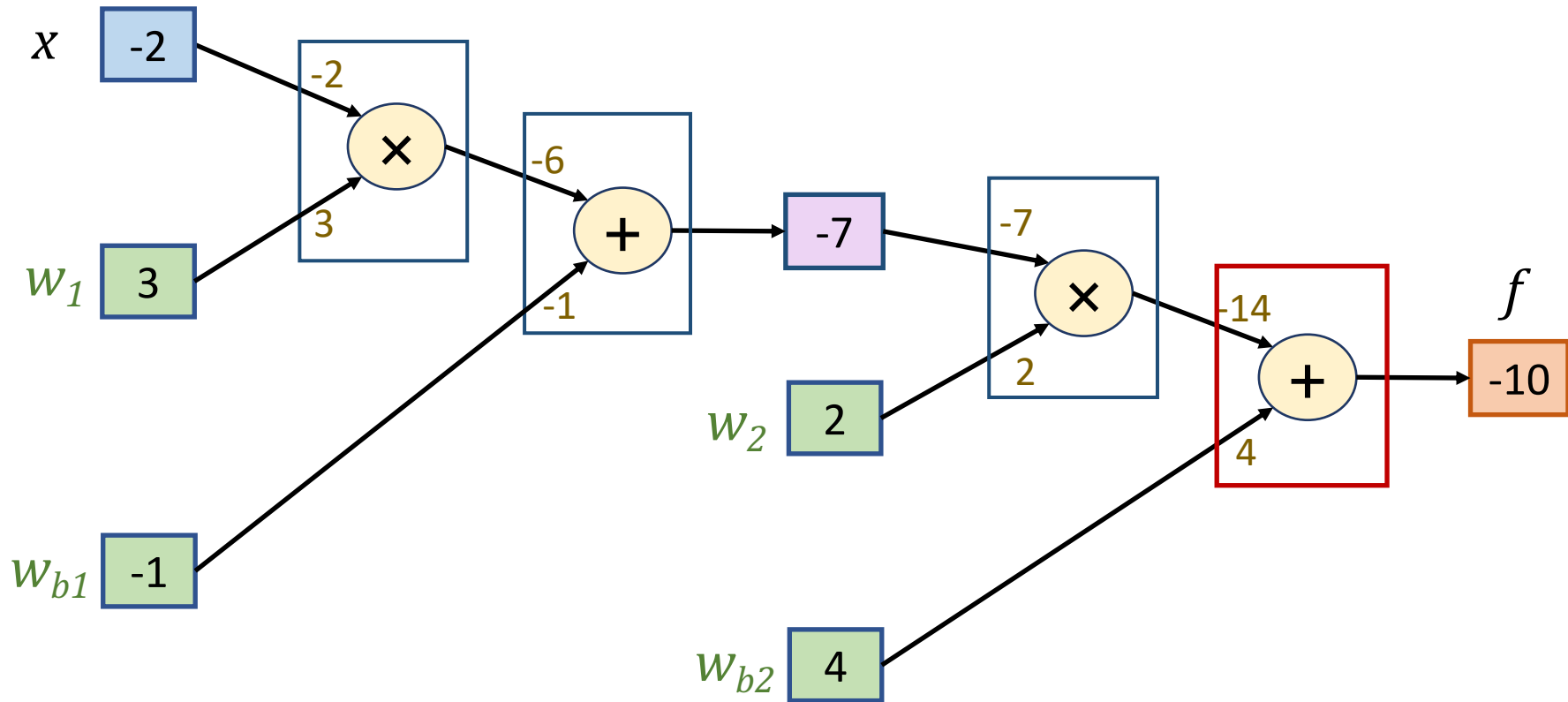
Network Training



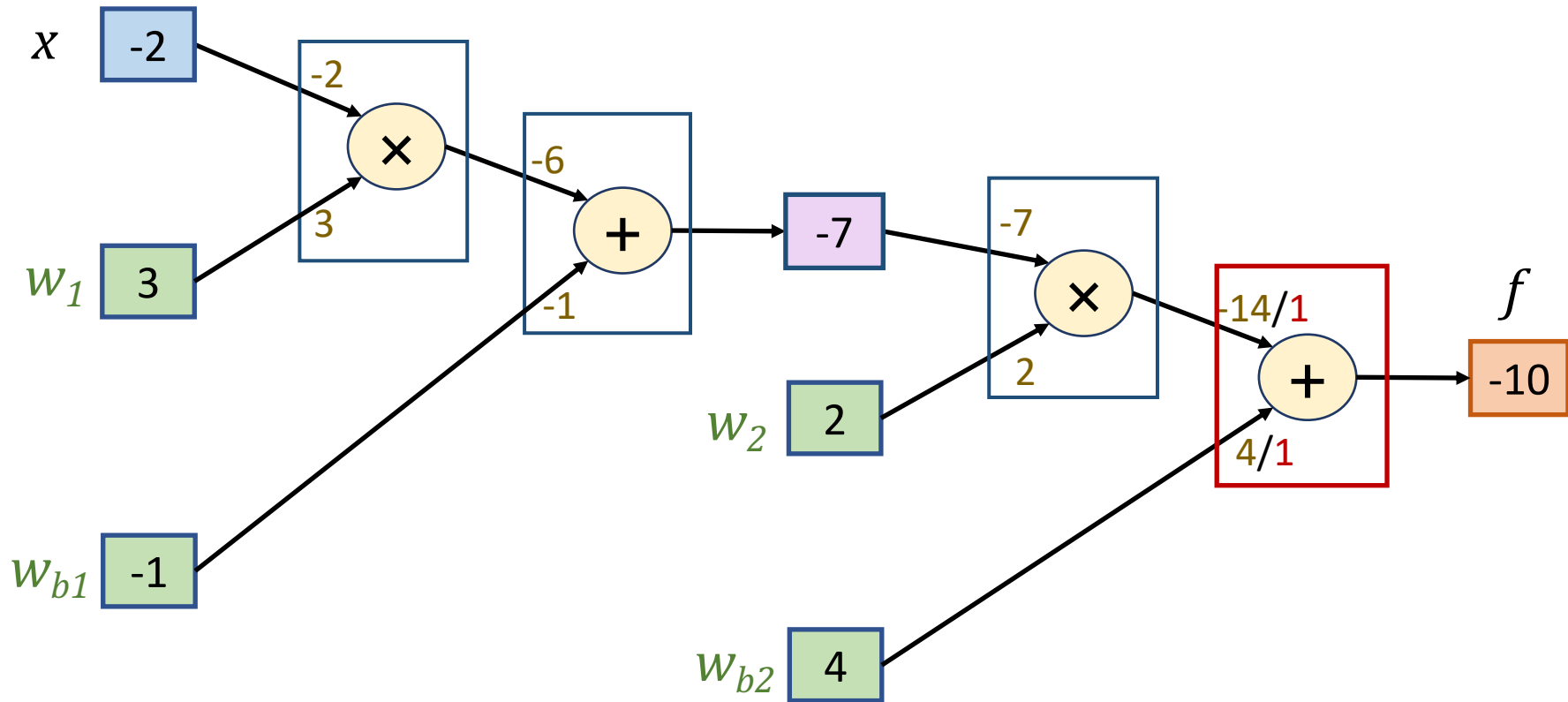
Network Training



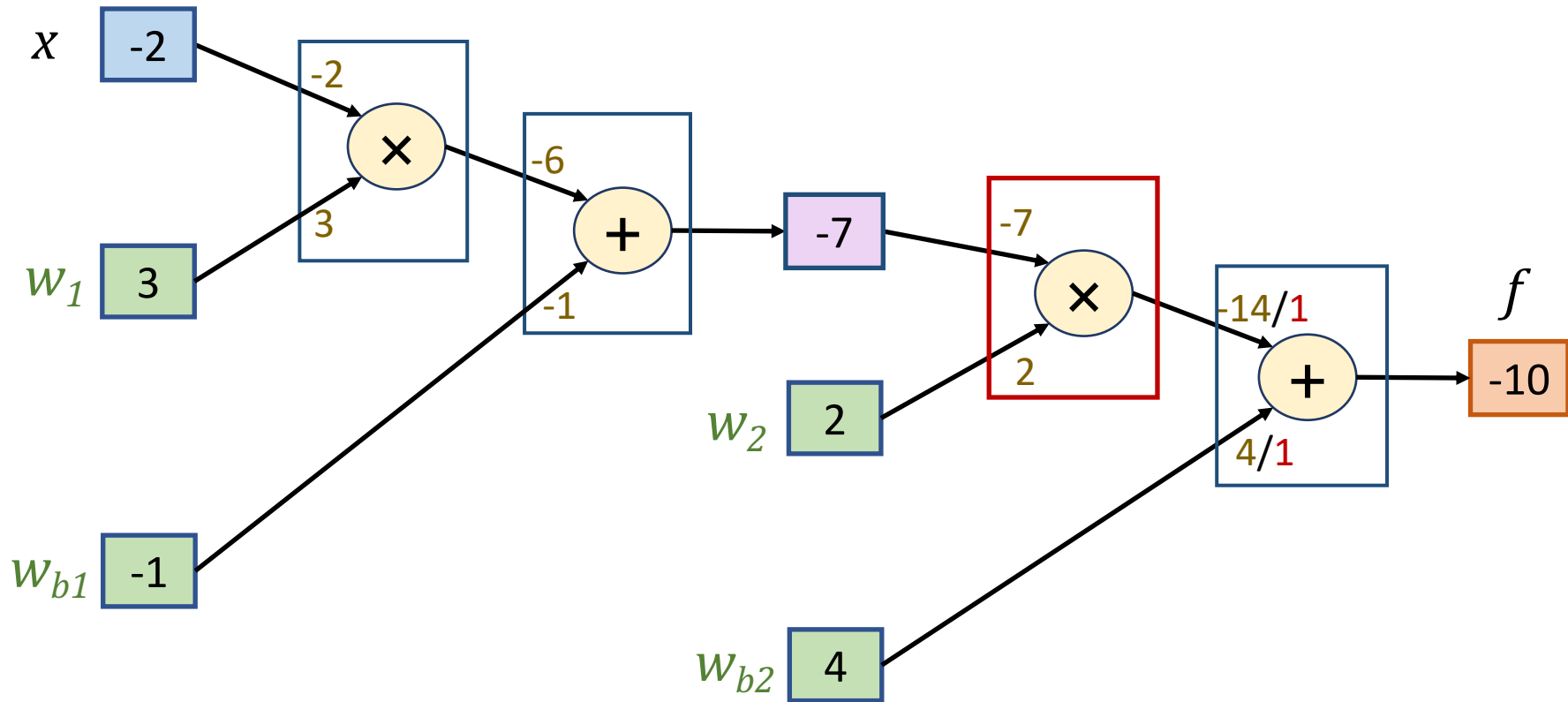
Network Training



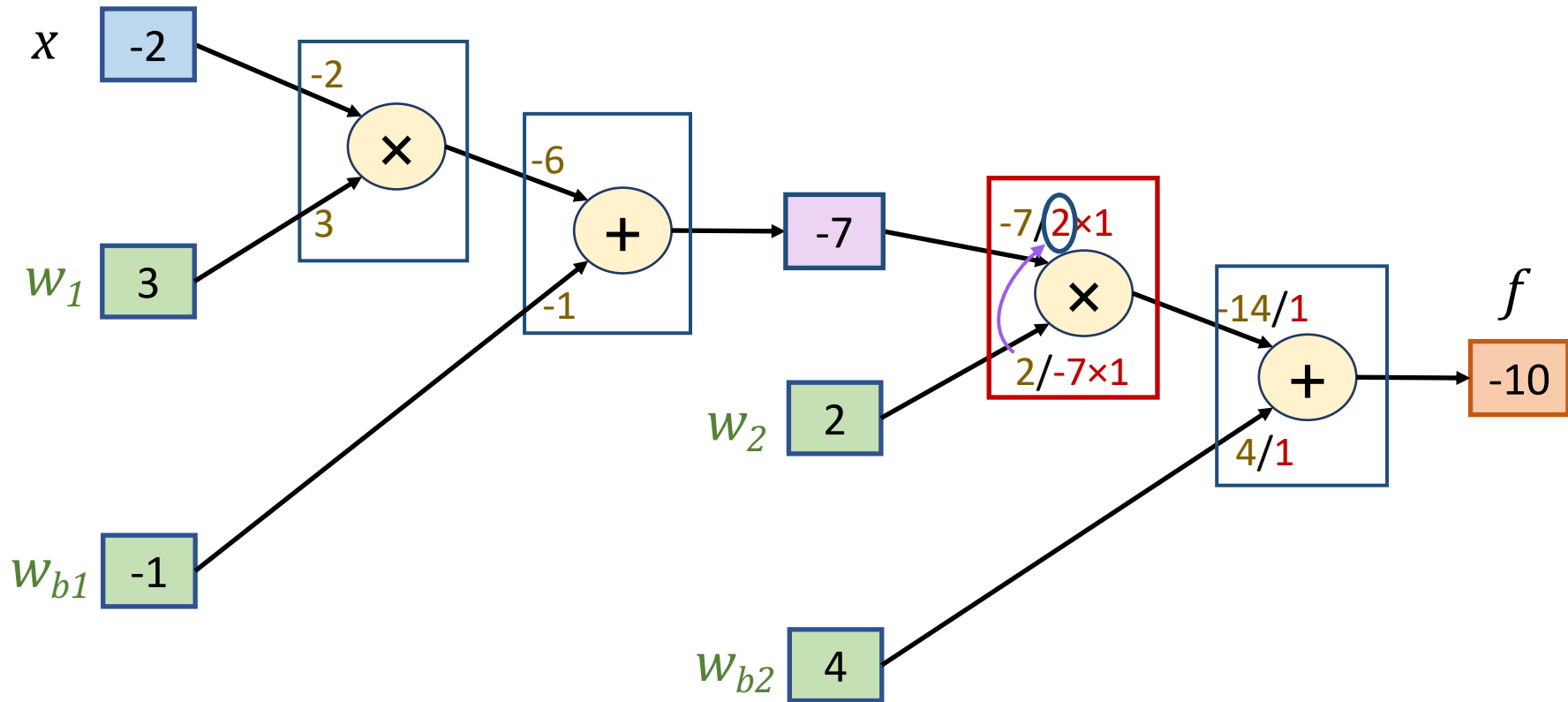
Network Training



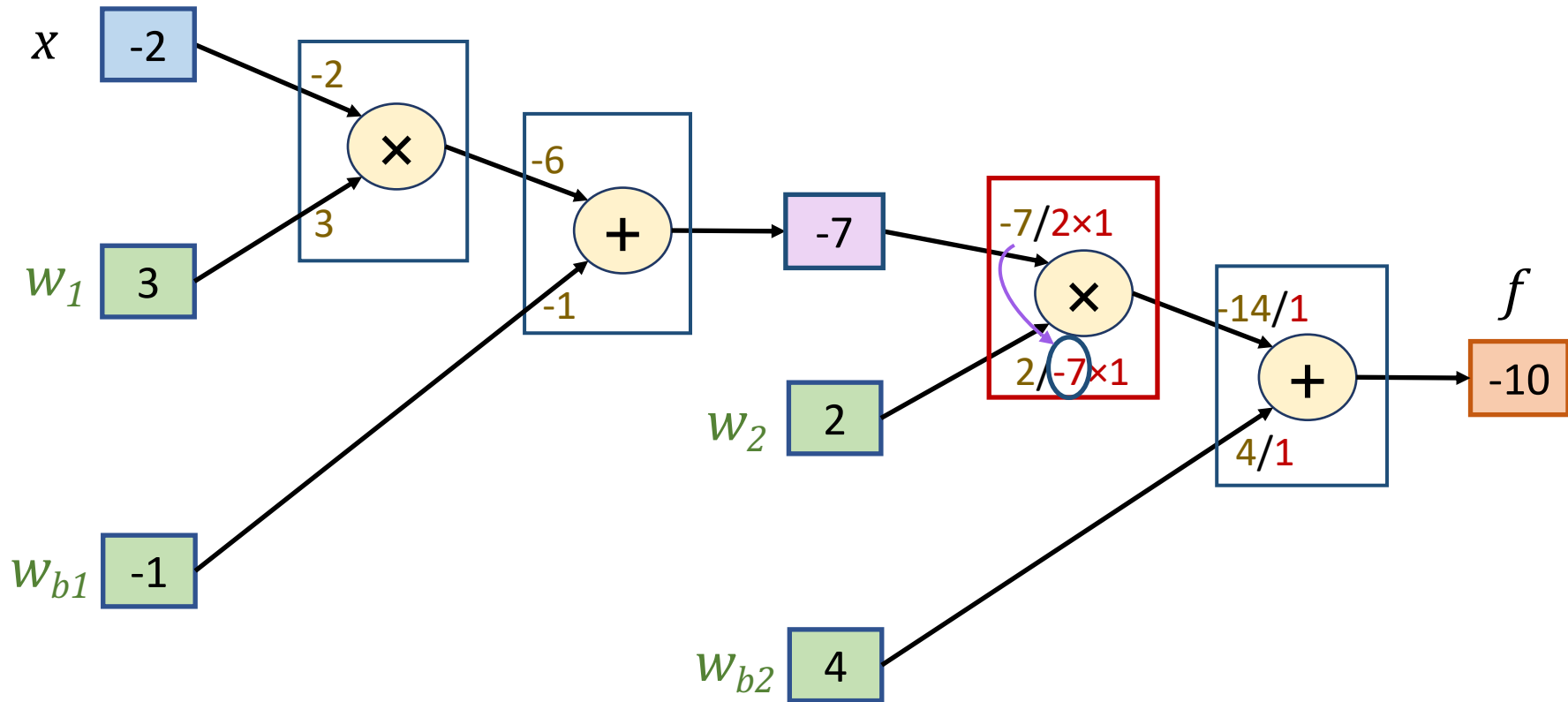
Network Training



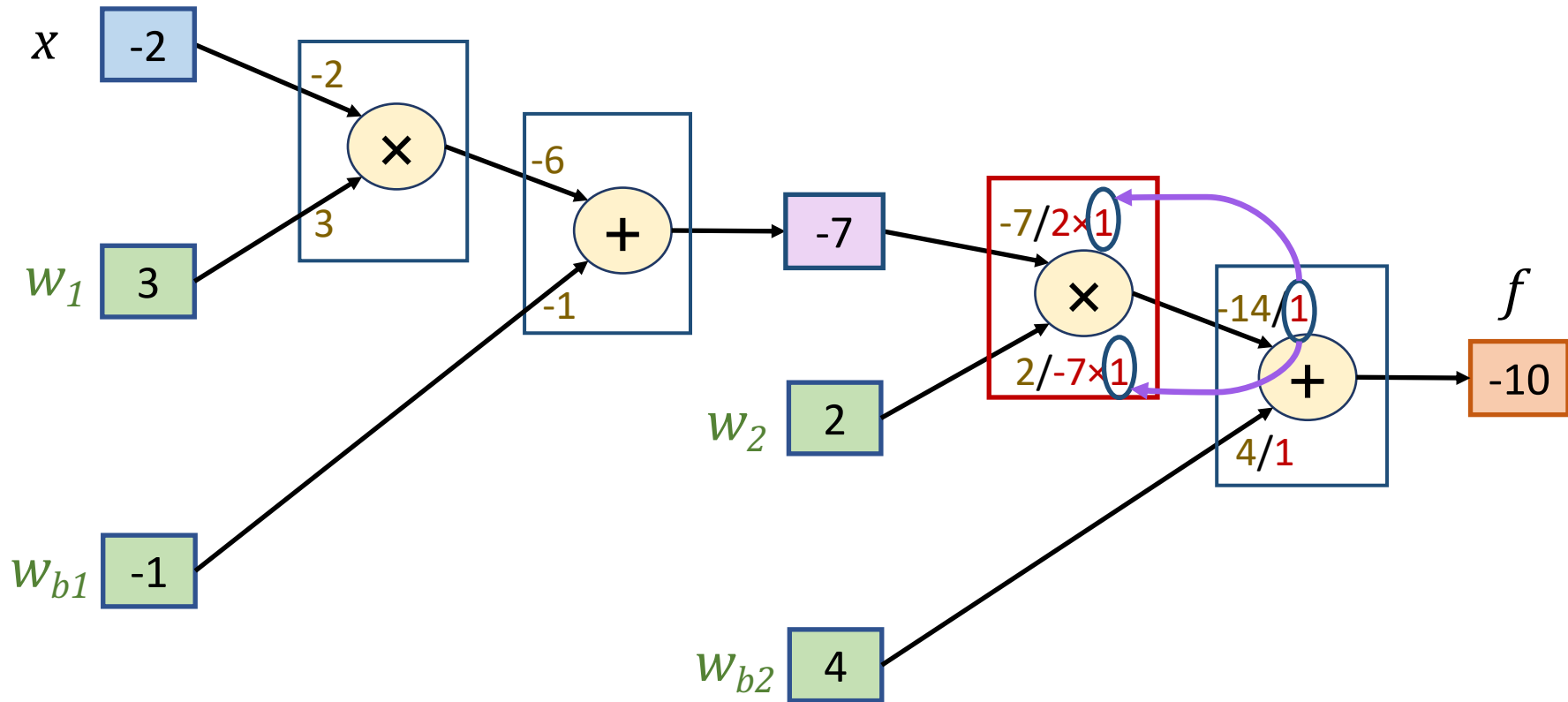
Network Training



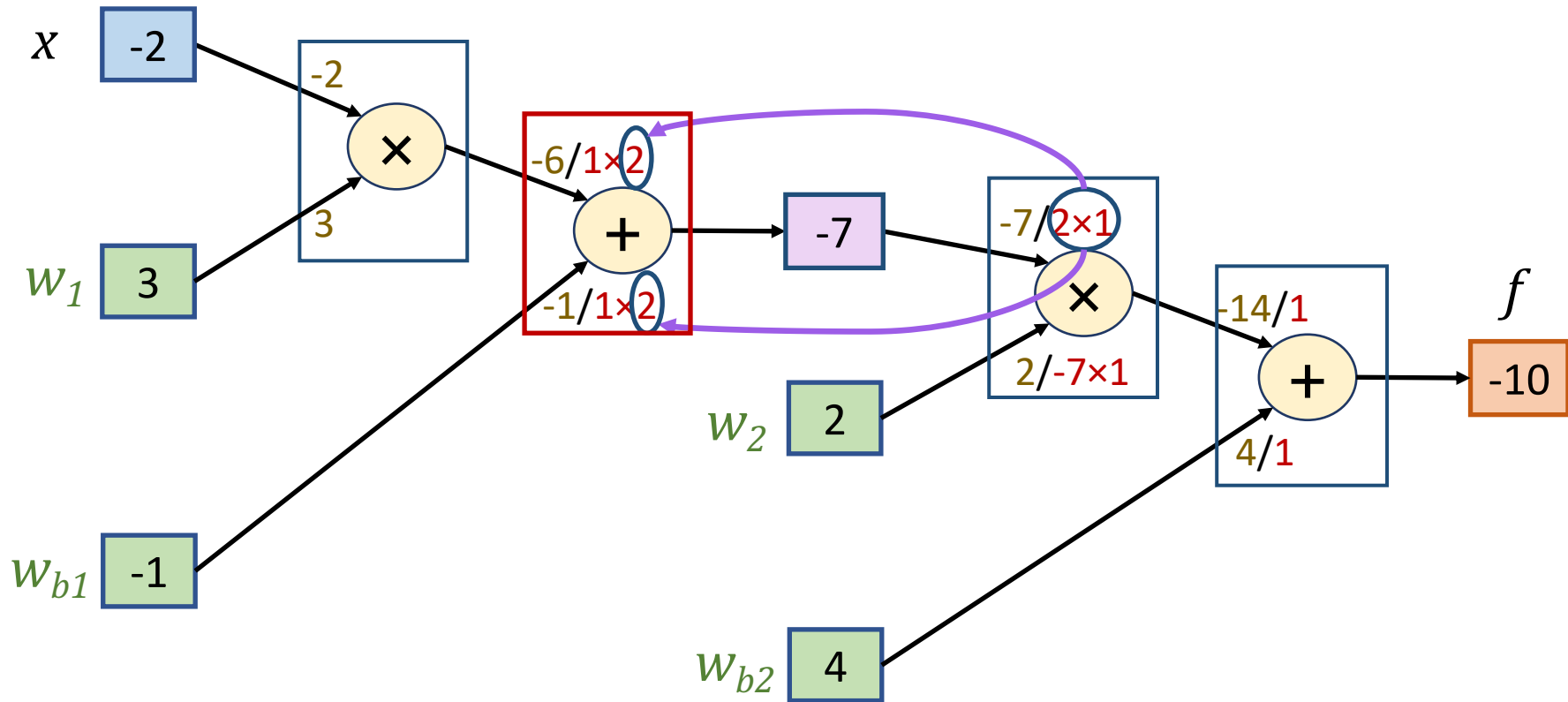
Network Training



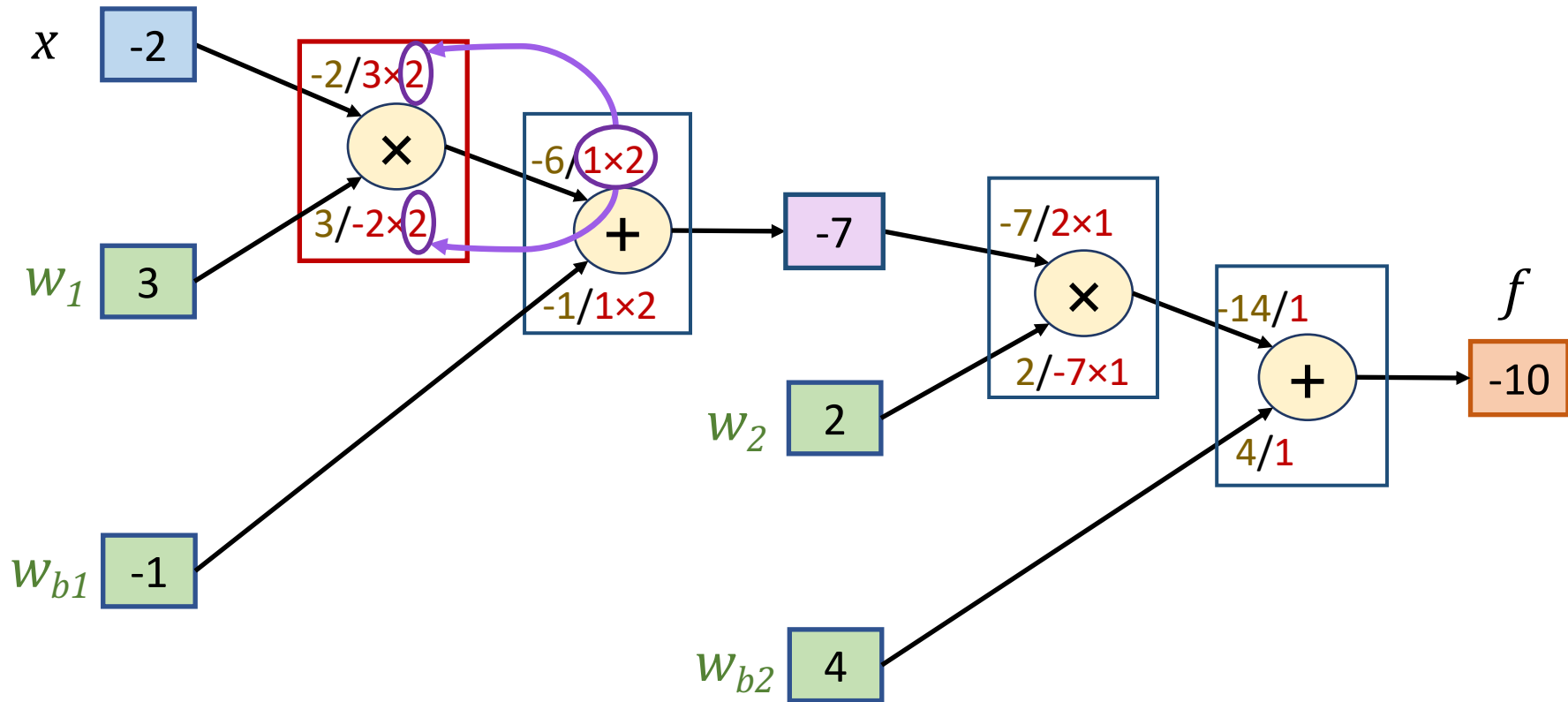
Network Training



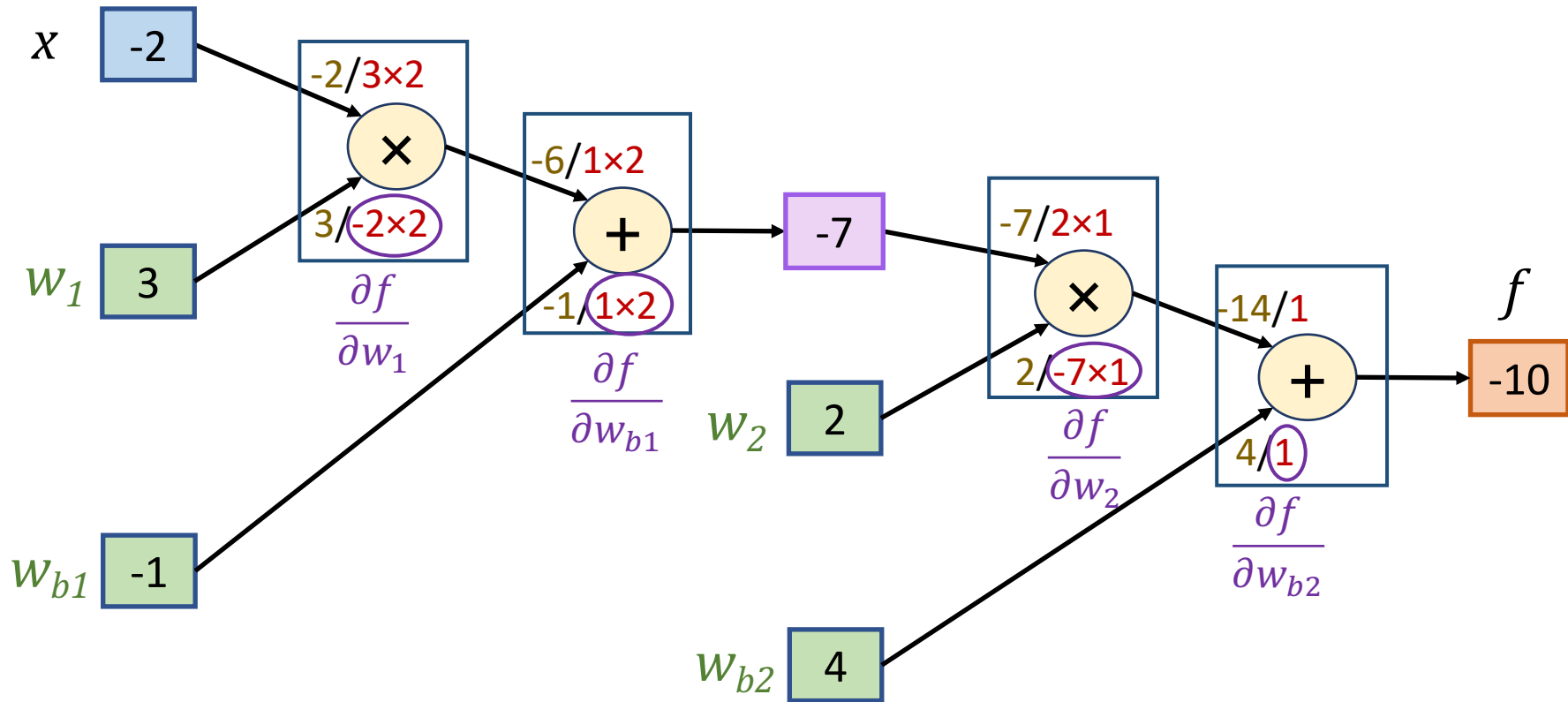
Network Training



Network Training



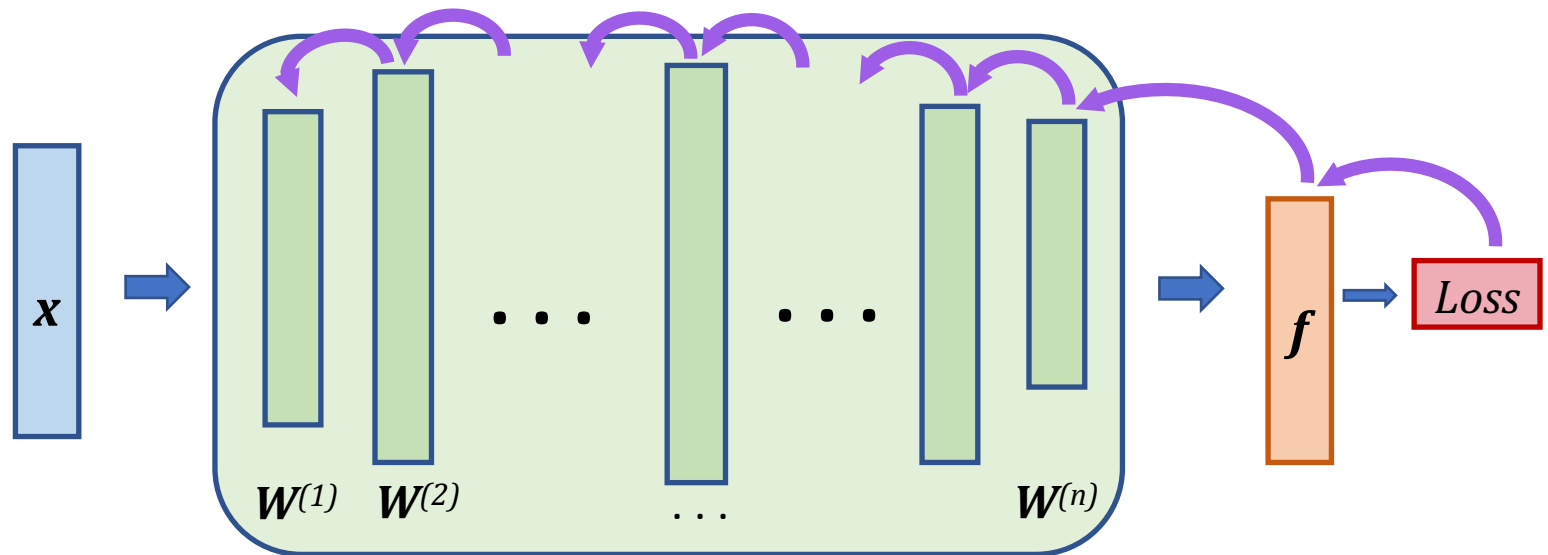
Network Training



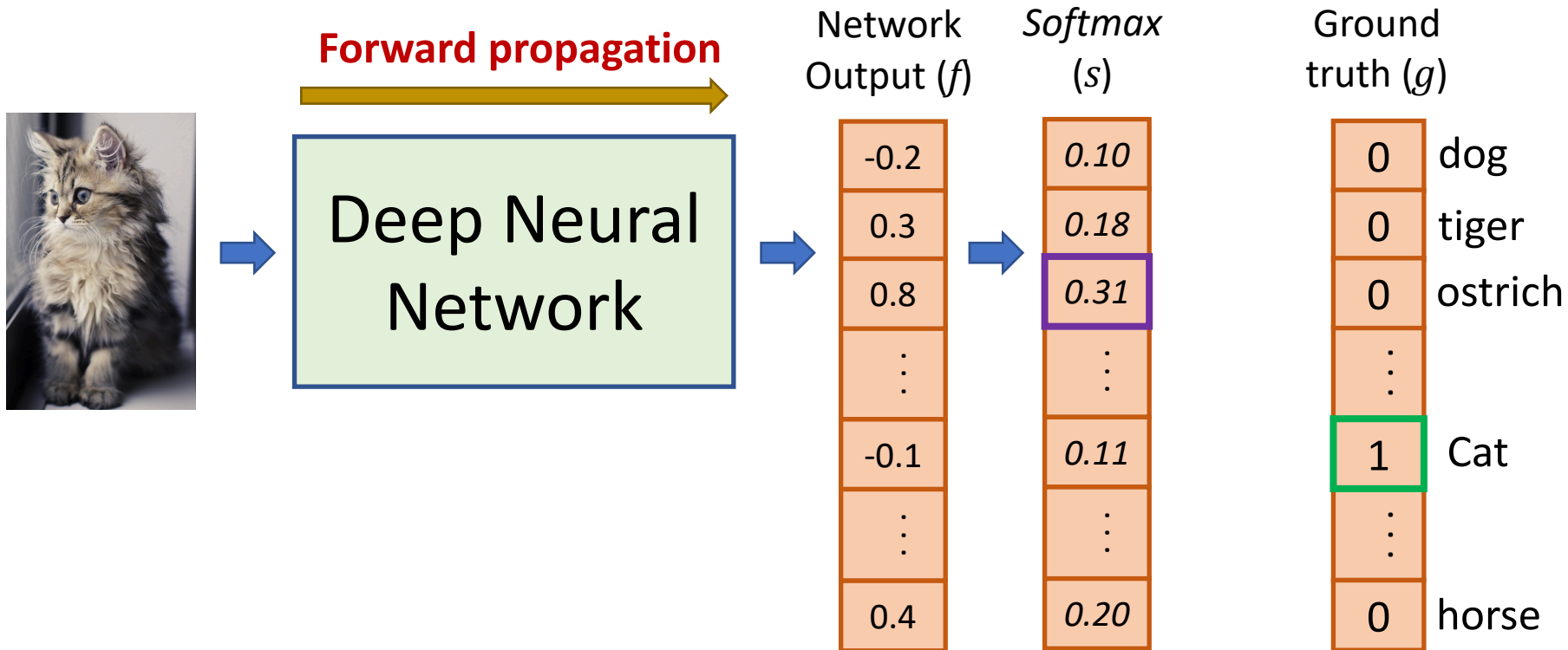
Network Training

We can add more layers as well as our loss function

All functions must be differentiable



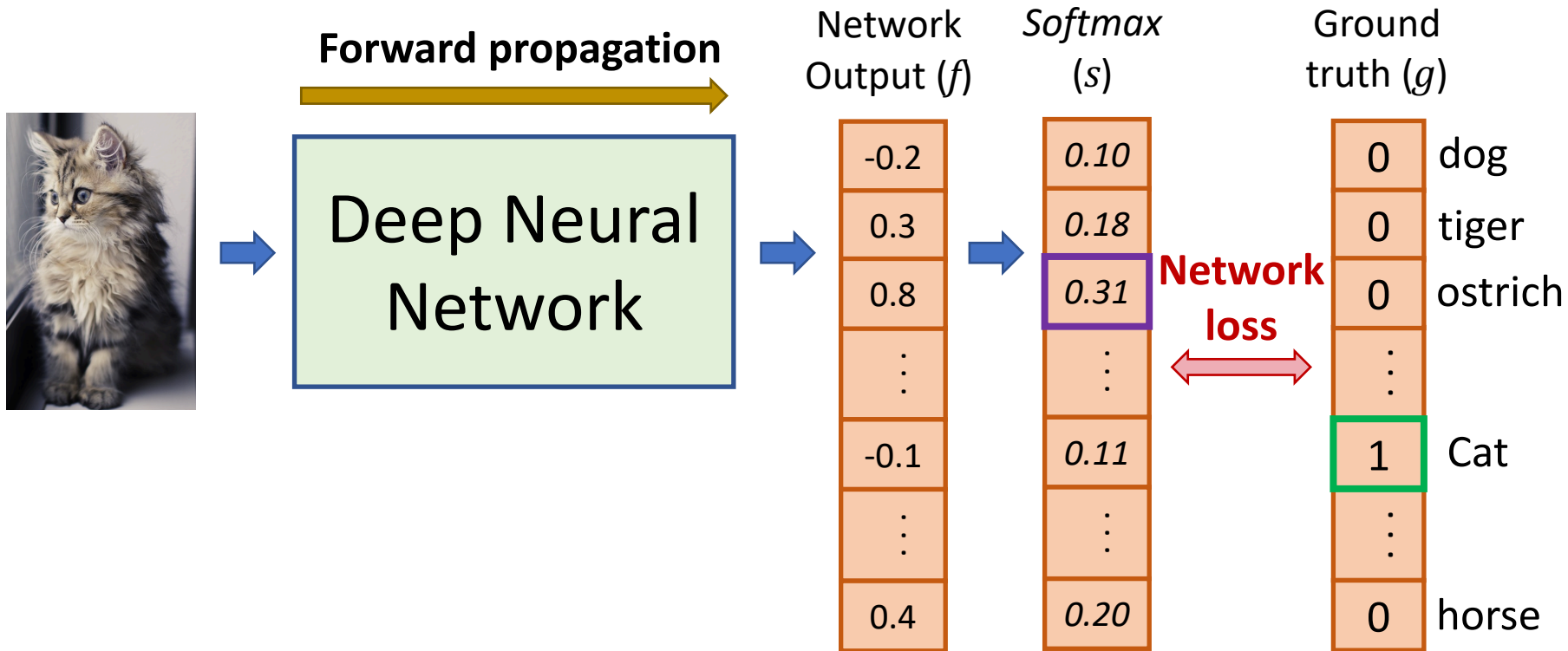
Summary and Conclusion



Optimisation during training

1. It makes a prediction using its current parameter set (weights).
2. We give it a feedback of how good its prediction was.
3. It updates (refines) its parameter set according to our feedback

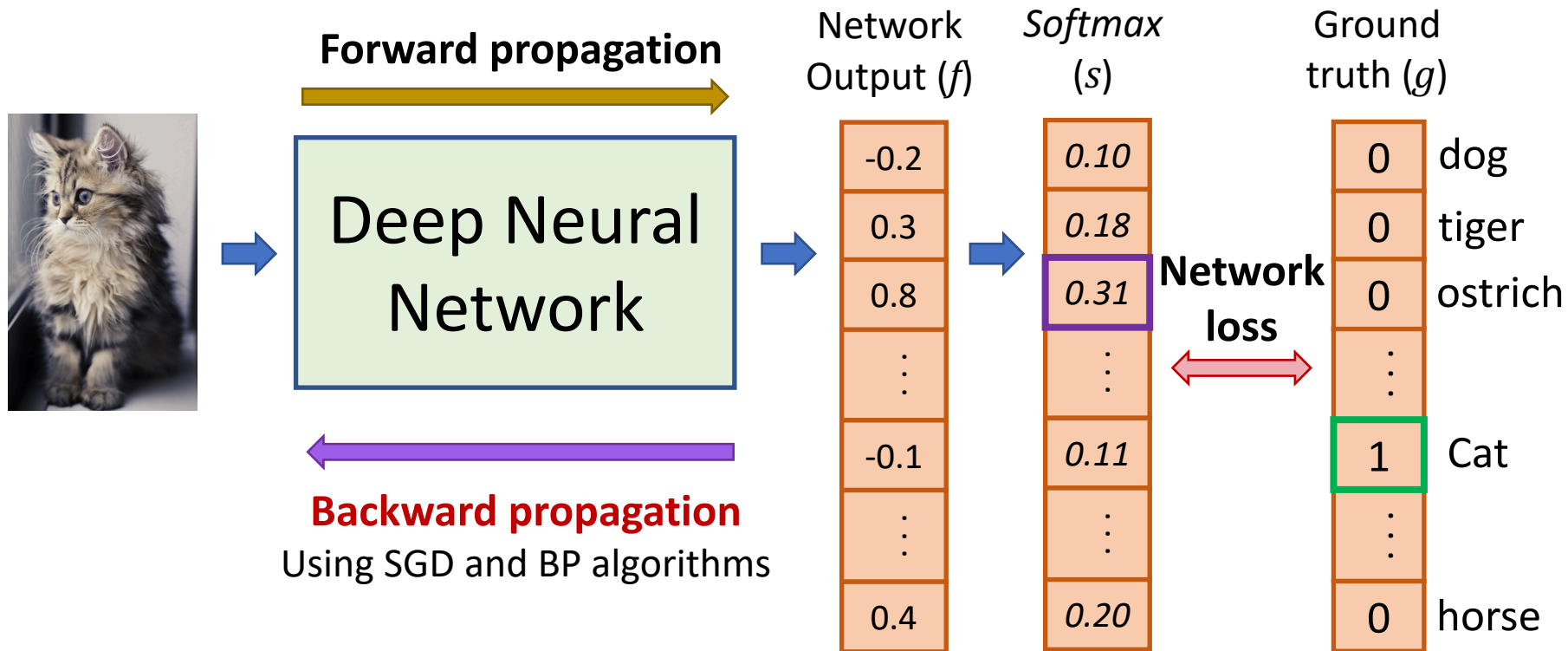
Summary and Conclusion



Optimisation during training

1. It makes a prediction using its current parameter set (weights).
2. We give it a feedback of how good its prediction was.
3. It updates (refines) its parameter set according to our feedback

Summary and Conclusion



Optimisation during training

1. It makes a prediction using its current parameter set (weights).
2. We give it a feedback of how good its prediction was.
3. It updates (refines) its parameter set according to our feedback

Summary and Conclusion

- Gradient Descent is an optimization algorithm used for minimizing the network loss
- It updates the parameter set by moving in the opposite direction of the gradient of the loss function
- Back Propagation is used along with Gradient Descent to update the weights in different layers of a network
- It uses chain rule recursively to back-propagate the loss through the network

Next part

We will discuss
different types of DNNs
and their architectures

Thanks for your attention